Fine-Tuning LLMs with Multi-GPU Training in an HPC System

NRIS

Norwegian research infrastructure services

Hicham Agueny, PhD
Scientific Computing Group
Info. Tech. Department, UiB/NRIS

Challenges with Fine-Tuning LLMs

Storage & Memory Bottlenecks

- GPT-3 (175B): checkpoint ≈ **350 GB (3.5TB for 10 tasks)**
- GPU memory requirement: **~1.2 TB**

What is Needed?

- **Optimization strategies** to overcome hardware limitations
- **High-Performance Computing (HPC)** with:
  - Powerful CPUs & GPUs
  - High-speed interconnects
- **Well-optimized software stack including e.g.**
  - Communication libraries
  - ML frameworks …etc

Why HPC Matters

- Without HPC systems, fine-tuning large LLMs (e.g., GPT-3 175B) on large datasets is nearly impossible.

Purpose of this Workshop

- Introduce **fine-tuning concepts**
- Provide **hands-on practice**
- Demonstrate **HPC-specific execution**

Workshop Program

09:30 – 10:15 | Introduction to Fine-Tuning & Optimization Strategies (30+15min)

- Overview of LLM and Fine-tuning
- Concepts: Parameter-efficient methods (LoRA, LoRA Dropout)
- Model merging

10:30 – 11:30 | Environment Setup & Configuration Overview (Olivia)

- Overview of Olivia Supercomputer
- Setting up training environment - **interactive session**
- Config file deep dive of Llama

Hands-On Session – Practical Fine-Tuning & Inference

Task 1: Question-Answering (Alpaca dataset)

12:30 – 13:30 | Fine-tuning on 1 GPU + Inference (60 min)
13:45 – 14:00 | Profiling (15 min)
14:00 – 14:45 | Fine-tuning on multiple GPUs + Inference (35 min)

Task 2: Choose your own experiment — open-ended experiment

15:00 – 15:15 | Wrap-up & Discussion (for early leavers)
15:15 – 16:00 | Hands-on continuity

- Try e.g. summarization (Xsum dataset) with LoRA vs. full fine-tuning.
- Compare single vs. multi-GPU scaling.
- Adjust config parameters (learning rate, batch size, epochs) and compare outputs.

Learning Outcomes

- Learn about concepts of LoRA and LoRA Dropout
- Apply LoRA fine-tuning to a LLaMA model.
- Set up PyTorch+extra packages using singularity on an HPC system
- Get familiar with training configuration files for LlaMA models.
- Run training jobs on a single GPU and scale up to multiple GPUs.
- Perform inference tasks such as QA and summarization.
- Monitor GPU usage and memory utilization & Profiling

Introduction to Fine-Tuning & Optimization Strategies

- Overview of LLM and Fine-tuning
- Concepts: Parameter-efficient methods (LoRA, LoRA Dropout)
- Model merging

Overview of Large Language Models (LLMs)

- **LLM** is a type of AI model trained on vast amounts of text data to understand and generate human-like language.

- **LLM scope** includes tasks such as:
  - Text generation
  - Text summarization
  - Question answering
  - Translation
  - Code generation

- **Fine-tuning** is the process of**:**
  - Adapting a pre-trained model (like an LLM)
  - Using **task-specific or domain-specific data**
  - Continuing training on a smaller, targeted dataset
  - Updating all the parameters of the pre-trained model.

- **Why Fine-Tuning ?** It enables:
  - Faster training (without re-training from scratch)
  - Better performance on domain-specific tasks (or specific datasets)
  - Reduced data requirements

Pre-training vs Fine-tuning

Aspect	Pre-training	Fine-tuning
Definition	Training on a vast amount of unlabelled text data	Adapting a pre-trained model to specific tasks
Data Requirement	Extensive and diverse unlabelled text data	Smaller, task-specific labelled data
Objective	Build general linguistic knowledge	Specialise model for specific tasks
Process	Data collection, training on large dataset, predict next word/sequence	Task-specific data collection, modify last layer for task, train on new dataset, generate output based on tasks
Model Modification	Entire model trained	Last layers adapted for new task
Computational Cost	High (large dataset, complex model)	Lower (smaller dataset, fine-tuning layers)
Training Duration	Weeks to months	Days to weeks
Purpose	General language understanding	Task-specific performance improvement
Examples	GPT, LLaMA 3	Fine-tuning LLaMA 3 for summarisation

Ultimate Guide to Fine-Tuning LLMs https://arxiv.org/pdf/2408.13296

Challenge of Fine-tuning LLM

Full fine-tuning (updating all parameters for each task) becomes expensive.

- **High GPU Memory:** Requires immense GPU memory

  (e.g., **1.2TB for GPT-3 175B –** trainable parameters)

- **High Storage Cost:** Storing instances of fine-tuned models for each task requires **massive storage**

  (e.g., 100 fine-tuned GPT-3 models would be **350GB (size of checkpoint)x100= 35TB**).

- **High computational cost for training: R**equire significant compute resources from days to weeks depending on the model size and dataset scale.

Need of optimizing fine-tuning LLM

LoRA https://arxiv.org/pdf/2106.09685

Type of Fine-Tuning Options

- **Task-Specific Fine-Tuning**: Adapt models for tasks like summarization, code generation, QA, and classification using tas
  - Key Models for text summarization and Q&A: BERTSUM, GPT-3, T5, BERT

- **Domain-Specific Fine-Tuning**: Customize models for domains like medical, legal, or financial text.
  - **Dataset in Medical domain:** MedQA, MedMCQA, LiveQA, MedicationQA, HealthSearchQA

- **PEFT (Parameter-Efficient Fine-Tuning)**: Use methods like LoRA to fine-tune efficiently with fewer trainable parameters.

- **HFT (Half Fine-Tuning)**: Update only half the model's parameters to balance learning and retention.

Introducing LoRA (Low-Rank Adaptation)

Adapted from: Ultimate Guide to Fine-Tuning LLMs https://arxiv.org/pdf/2408.13296

LoRA Overview

LoRA: Low-Rank Adaptation of Large Language Models

Edward Hu* **Yelong Shen*** **Phillip Wallis** **Zeyuan Allen-Zhu**
Yuanzhi Li **Shean Wang** **Lu Wang** **Weizhu Chen**
Microsoft Corporation
{edwardhu, yeshe, phwallis, zeyuana,
yuanzhil, swang, luw, wzchen}@microsoft.com
yuanzhil@andrew.cmu.edu
(Version 2)

- **Core Idea:** Freeze the pre-trained model weights (the main model's weights do not change during fine-tuning)
  Inject trainable rank decomposition matrices into each layer of the Transformer architecture.

- **Concept:** Only **train low rank** matrices **A & B**, while keeping the **pre-trained weight matrix frozen**.

Modified forward pass

$$h = W_0 x + \Delta W x = W_0 x + BAx.$$

$W_0 \in \mathbb{R}^{n_1 \times n_2}$ Pre-trained weight matrix (frozen)

$A \in \mathbb{R}^{r \times n_2}$

$B \in \mathbb{R}^{n_1 \times r}$

Low-rank update (trainable)

x Input vector

h Output of the forward pass

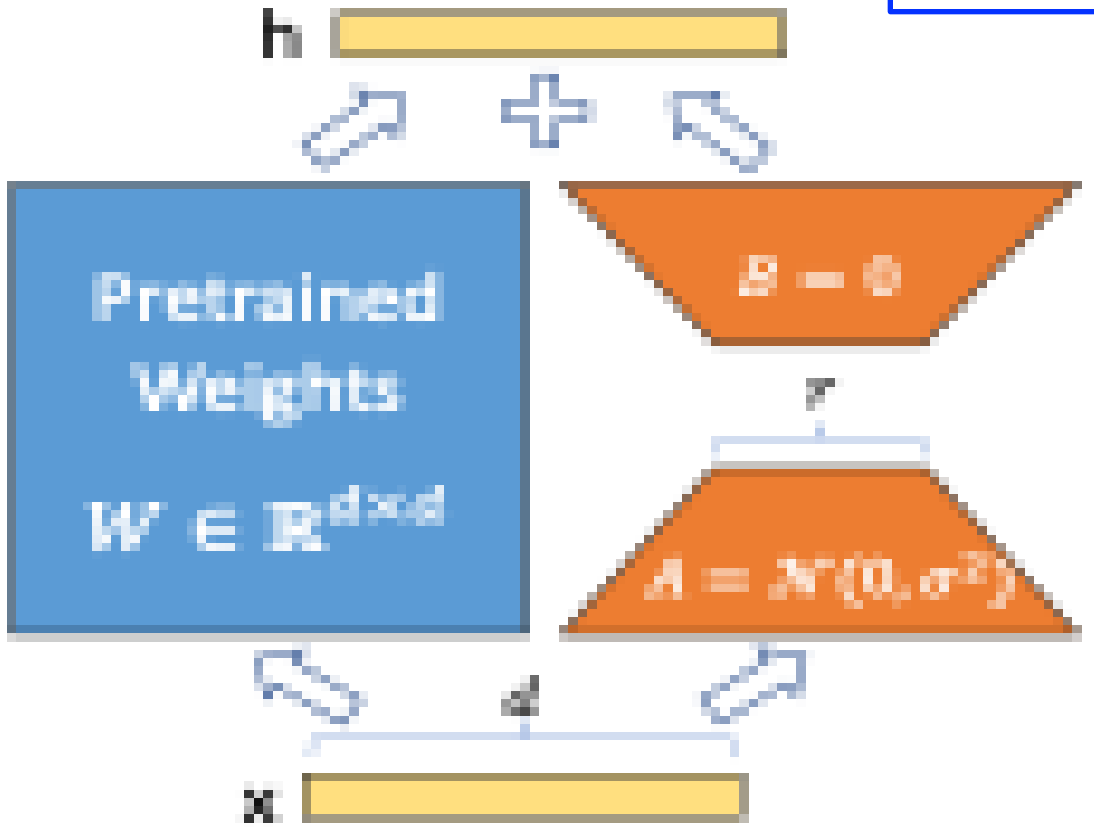
$\Delta W = BA$ Low rank decomposition

LoRA https://arxiv.org/pdf/2106.09685

LoRA Overview

LoRA: Low-Rank Adaptation of Large Language Models

Edward Hu* **Yelong Shen*** **Phillip Wallis** **Zeyuan Allen-Zhu**
Yuanzhi Li **Shean Wang** **Lu Wang** **Weizhu Chen**
Microsoft Corporation
{edwardhu, yeshe, phwallis, zeyuana,
yuanzhil, swang, luw, wzchen}@microsoft.com
yuanzhil@andrew.cmu.edu
(Version 2)



Modified forward pass

$$h = W_0 x + \Delta W x = W_0 x + B A x.$$

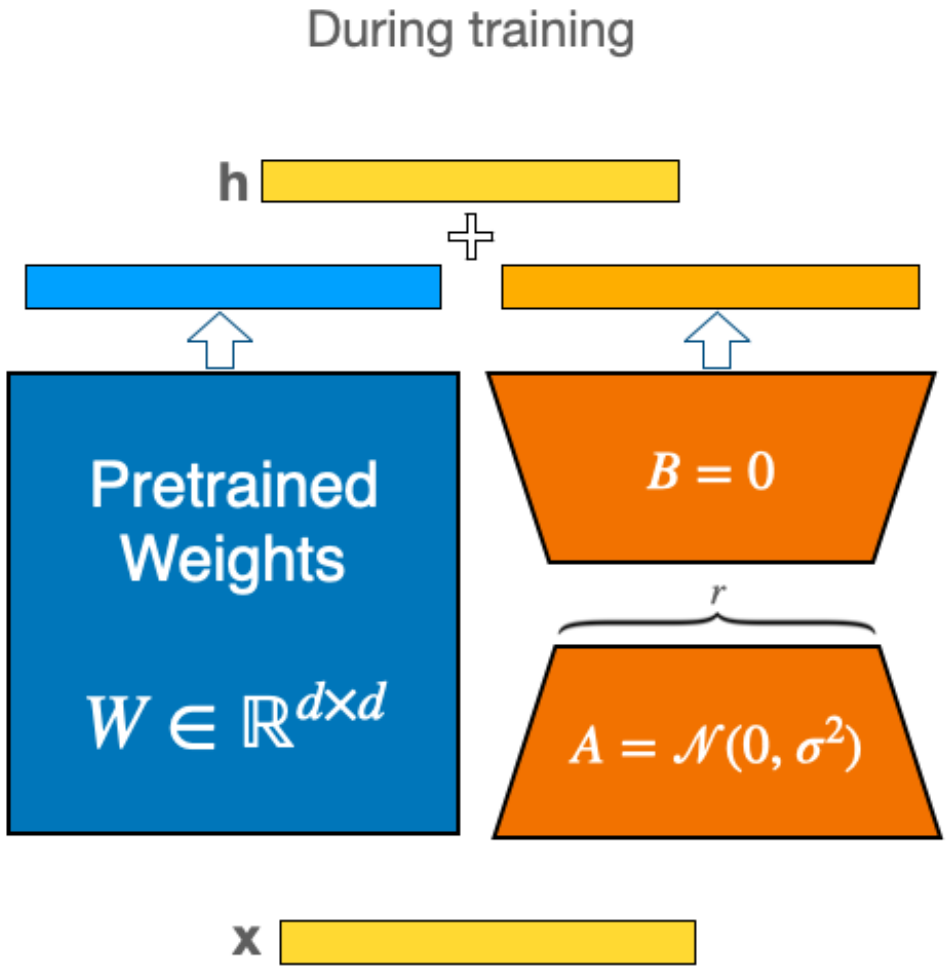
Random Gaussian initialization for A and zero for B

$\Delta W = BA$ is zero at the beginning of training

LoRA https://arxiv.org/pdf/2106.09685

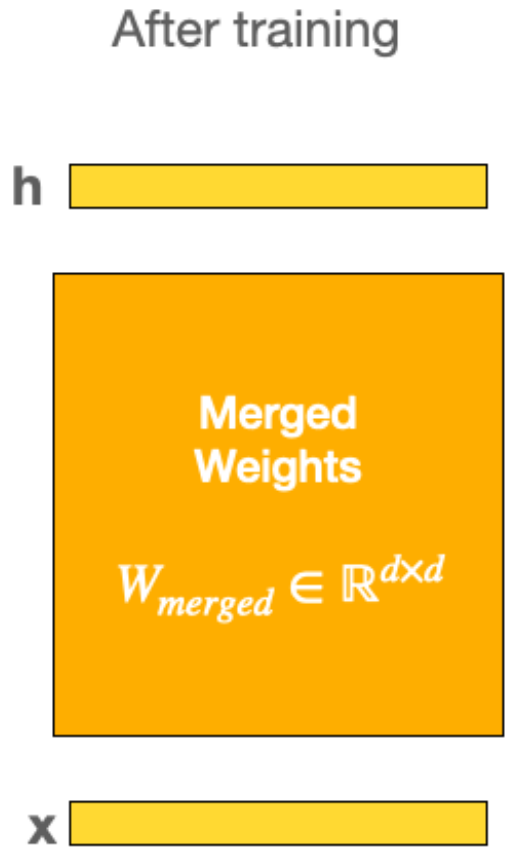
Merging LoRA weights into the base model

- Training **multiple tasks** (e.g. QA, summarization etc.) requires **large storage space to store checkpoints.**
- **Model merging** is a solution to storage space related challenge.



$$h = Wx + BAx$$

$$h = \underbrace{(W + BA)}_{W_{merged}}x$$



https://huggingface.co/docs/peft/en/developer_guides/lora#weight-decomposed-low-rank-adaptation-dora

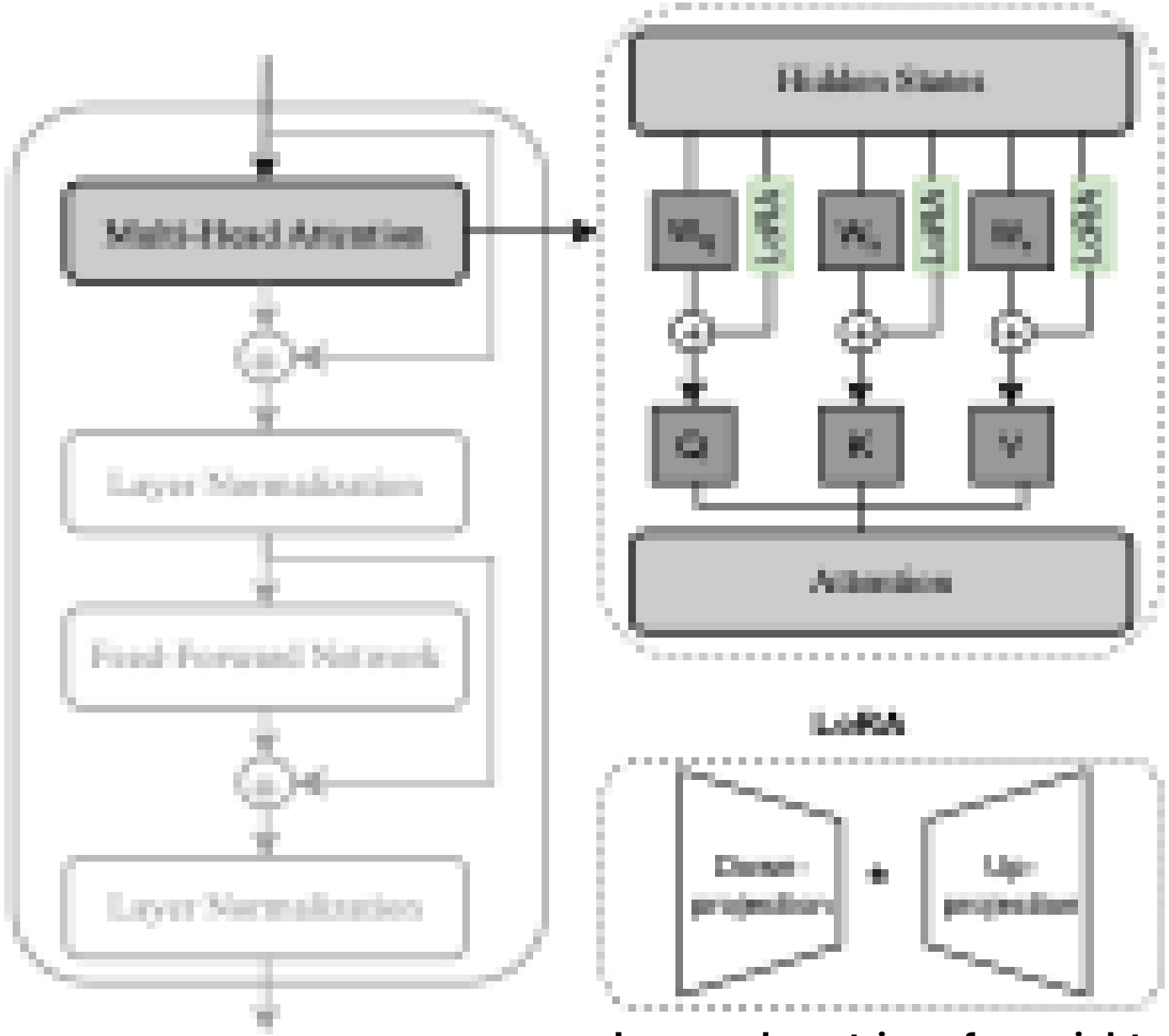
LoRA implementation

- **In Transformer, Each attention head**: transforms (linear) the **input embeddings** into: **Query (Q), Key (K)** and **Value (V)** matrices.

  - **"Query"** vector becomes a learned representation of what information a token is looking for.
  - **"Key"** vector represents what information the token contains.
  - **"Value"** vector contains the actual content to be aggregated.

Fastformer https://arxiv.org/pdf/2312.12148

Architecture of
a **Transformer** block

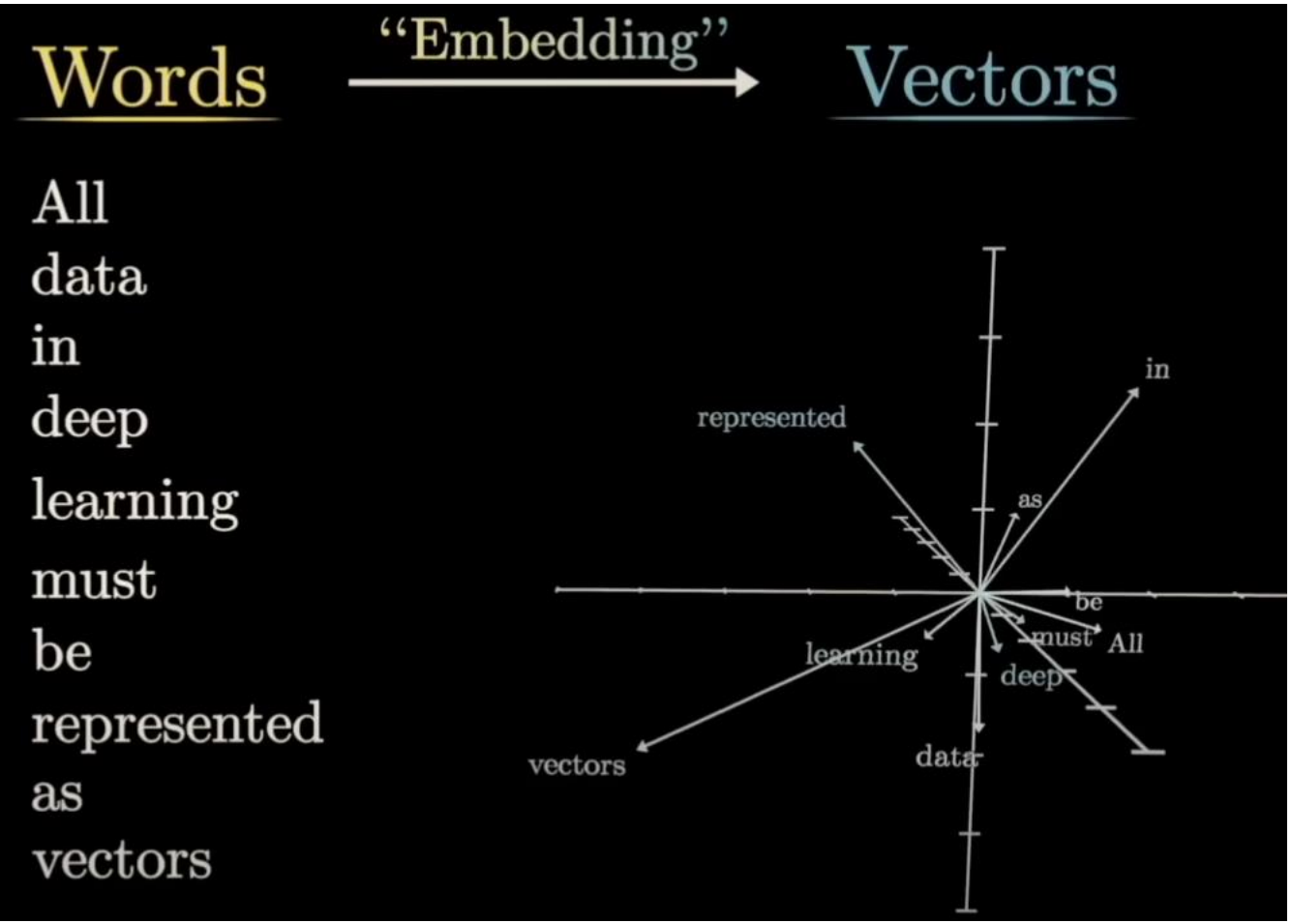


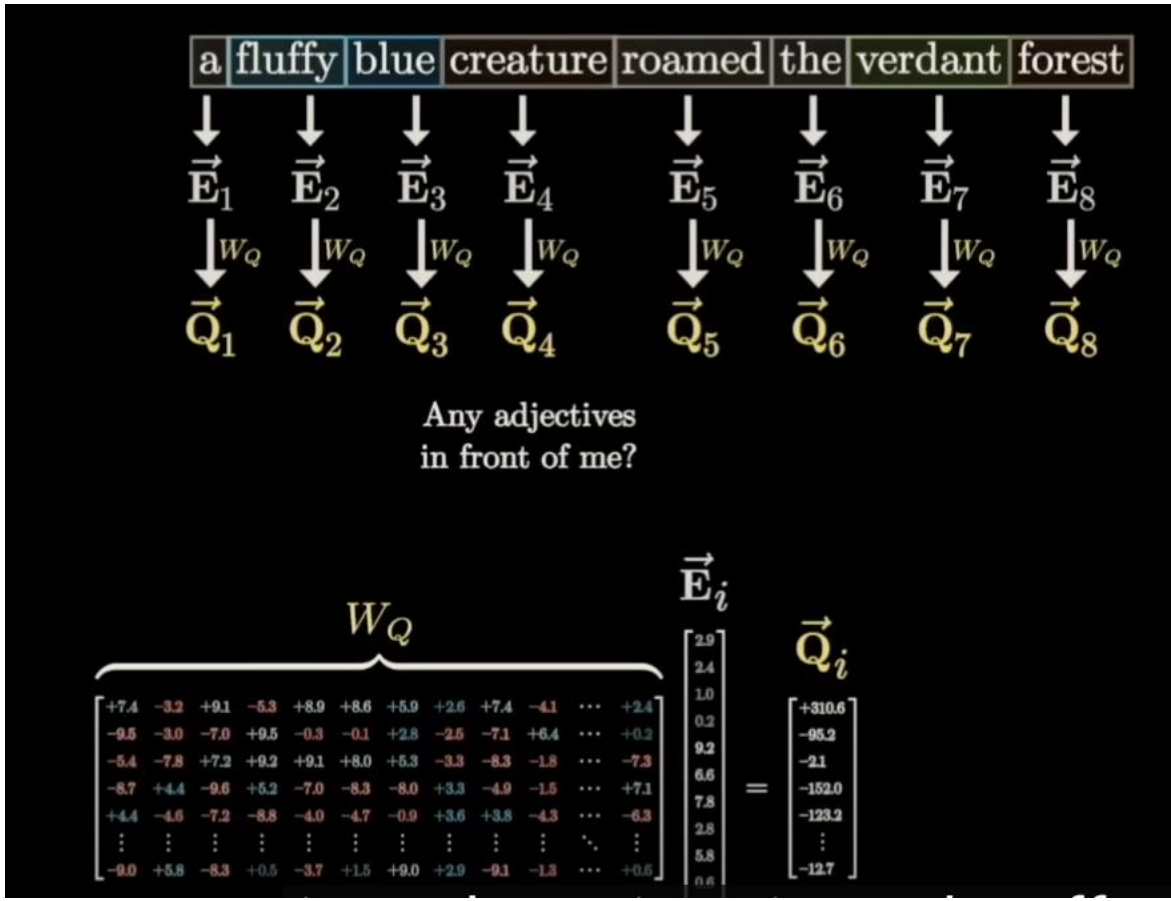
Low-rank matrices for weights

PEFT https://arxiv.org/pdf/2312.12148

Word Representations in Vector Space

Associating words with vectors (embedding)



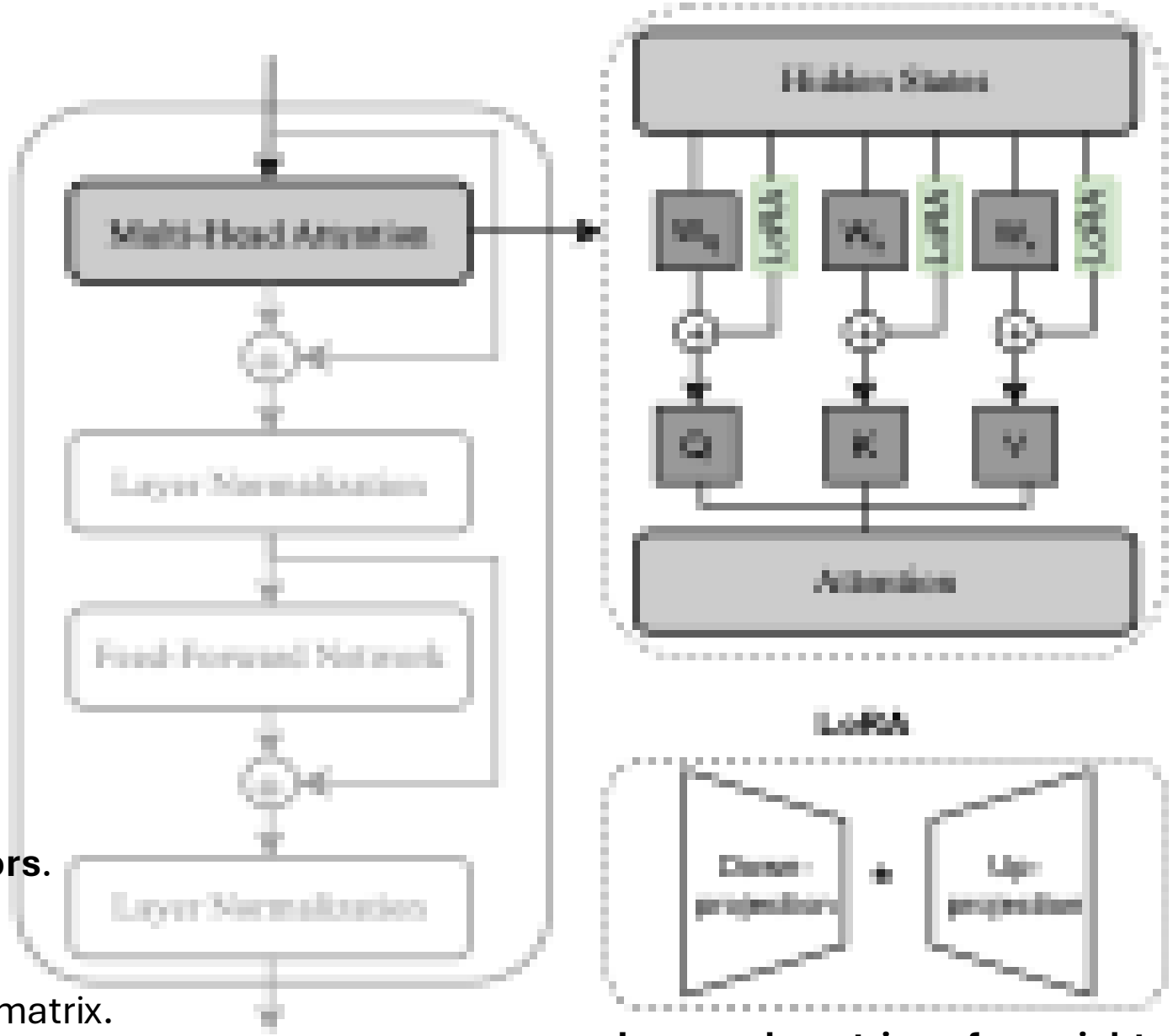


- **Ei** = word embedding for token i.
- **WQ** = learned weight matrix that converts embeddings into queries.
- **Qi** = query vector, representing what the word wants to attend to.

Visualizing transformers and attention | Talk for TNG Big Tech Day '24
https://www.youtube.com/watch?v=KJtZARuO3JY&t=155s
Efficient Estimation of Word Representations in Vector Space
https://arxiv.org/pdf/1301.3781

LoRA implementation

Architecture of
a **Transformer** block



- **In Transformer, Each attention head**:
  transforms the input (words) into:
  Query (Q), Key (K) and **Value (V)** matrices.

- **LoRA** is implemented inside each **Head Attention.**

- The **LoRA matrices** are **injected into the attention layer's weight matrices**, specifically,
  Wq and Wv projection matrices.

$$W'_Q = W_Q + \Delta W_Q, \quad \Delta W_Q = B_Q A_Q$$

$$W'_V = W_V + \Delta W_V, \quad \Delta W_V = B_V A_V$$

Wq transforms the embedding vectors into **Query vectors**.

- In **LoRA**, **Wq, Wk, Wv, are frozen** pre-trained weight matrix.

Low-rank matrices for weights

Fastformer https://arxiv.org/pdf/2312.12148

PEFT https://arxiv.org/pdf/2312.12148

Advantages of LoRA

- **Massive Parameter Reduction:** Reduces trainable parameters significantly
  (e.g., 10,000 times for GPT-3 175B compared to fine-tuning).
- **Reduced GPU Memory:** Lowers GPU memory requirements
  (e.g., 3 times less VRAM for GPT-3 175B, from 1.2TB to 350GB).
- **Reduced storage cost:** e.g. GPT-3 175B reduce checkpoint size from 350 GB to 35 MB for LoRA weights.
- **Improved Training Throughput:** Faster training due to fewer parameters needing gradient calculation and optimizer states.
  (e.g., 25% speedup on GPT-3 175B).
- **Efficient Task Switching:** Rapidly switch between tasks by simply swapping the small LoRA matrices,
  instead of reloading the entire model.
- **No Inference Latency:** Trainable LoRA matrices (A and B) can be merged with the frozen pre-trained weights.

LoRA https://arxiv.org/pdf/2106.09685

Why LoRA seems to work ?

Model&Method	# Trainable Parameters	WikiSQL Acc. (%)	MNLI-m Acc. (%)	SAMSum R1/R2/RL
GPT-3 (FT)	175,255.8M	**73.8**	89.5	52.0/28.0/44.5
[redacted]				
GPT-3 (LoRA)	4.7M	73.4	**91.7**	**53.8/29.8/45.9**
GPT-3 (LoRA)	37.7M	**74.0**	**91.6**	53.4/29.2/45.1

- **Low intrinsic rank hypothesis**: Tactual update required to adapt to a new task lies in a much smaller subspace.
  No need for updating the pre-trained weights.
  Key information is captured even with low ranks.

- **Empirical experiments**: Tests show that even very small LoRA ranks can match full fine-tuning performance.

LoRA https://arxiv.org/pdf/2106.09685

Risk of fine-tuning using LoRA

LoRA requires optimization of low rank for optimal performance.

→ **Risk of overfitting on the training dataset**
(affects the effectiveness of the fine-tuned model)

LoRA Dropout is a mechanism designed to overcome overfitting in LoRA-based methods

---

LoRA Dropout as a Sparsity Regularizer for Overfitting Control

---

Yang Lin[1 2*] Xinyu Ma[1 2*] Xu Chu[1 2] Yujie Jin[1 2] Zhibang Yang[2] Yasha Wang[1 2] Hong Mei[1]

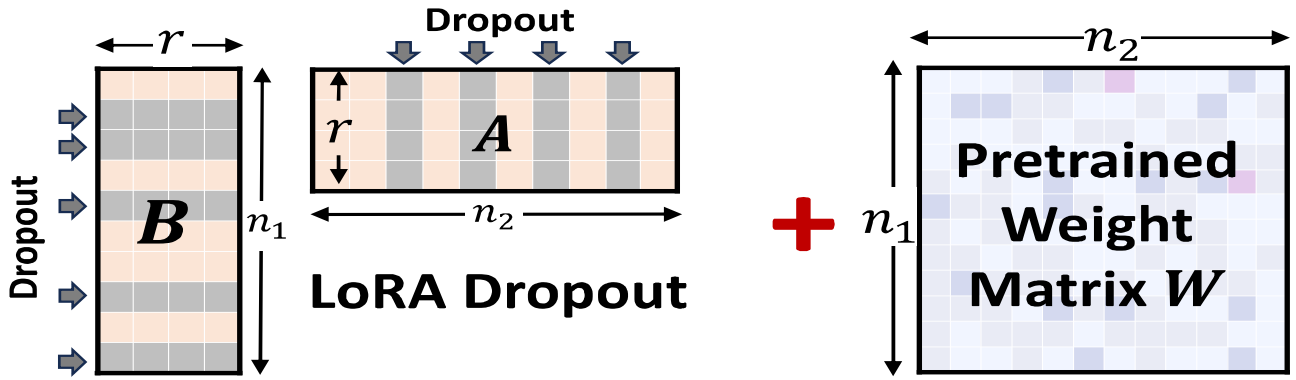
LoRA Dropout as a Sparsity Regularizer for Overfitting Control, https://arxiv.org/pdf/2404.09610

Concept of LoRA Dropout

Randomly drop rows and columns from both tunable low-rank parameter matrices:

$$\hat{A} = A \cdot \mathrm{diag}(m_A), m_A \sim \mathrm{Bern}(1-p);$$

$$\hat{B} = \left(B^\top \cdot \mathrm{diag}(m_B)\right)^\top, m_B \sim \mathrm{Bern}(1-p),$$



Drop column in A: Zero out a column → that dimension is ignored

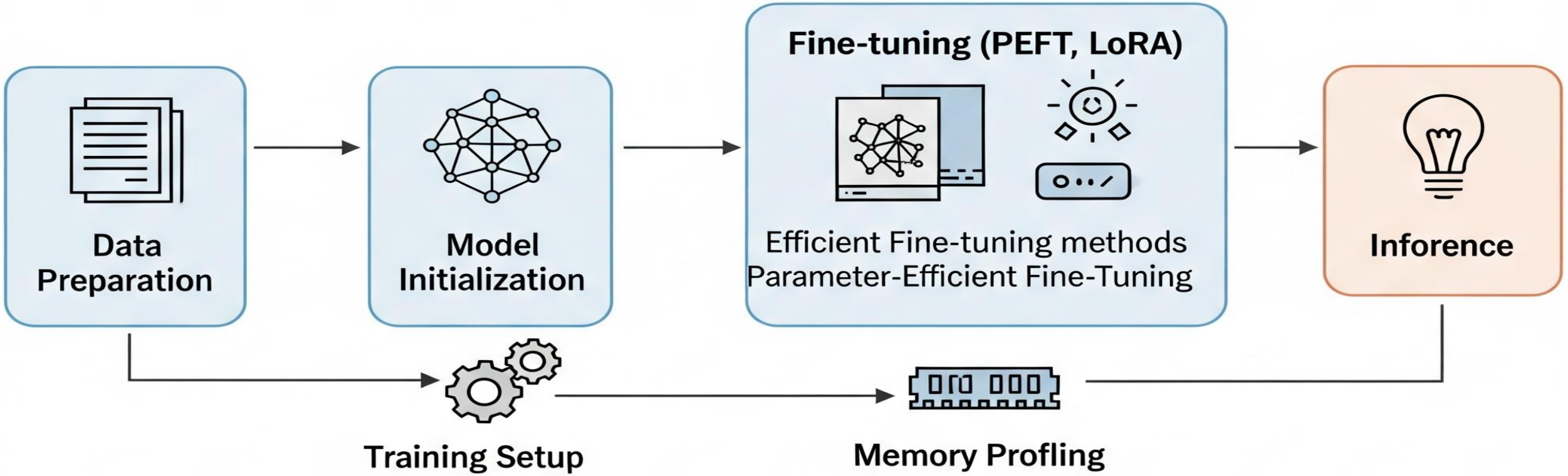
- p=0, no dropout, which means the original matrix is unchanged.

$$A \cdot \mathrm{diag}(m_A) = A \cdot I = A$$

Method	#Params	MNLI M-Acc	SST-2 Acc	CoLA Mcc	QQP Acc	QNLI Acc	RTE Acc	MRPC Acc	STS-B Corr	All Avg.
Full Fine-Tuning	184M	89.90	95.63	69.19	**92.40**	94.03	83.75	89.46	91.60	88.25
$\mathrm{LoRA}_{r=8}$	1.33M	90.65	94.95	69.82	91.99	93.87	85.20	89.95	91.60	88.50
LoRA+Dropout	1.33M	**90.85**	95.87	71.32	92.22	**94.56**	88.09	**91.42**	92.00	89.54

LoRA Dropout as a Sparsity Regularizer for Overfitting Control, https://arxiv.org/pdf/2404.09610

Fine-tuning pipeline for LLM

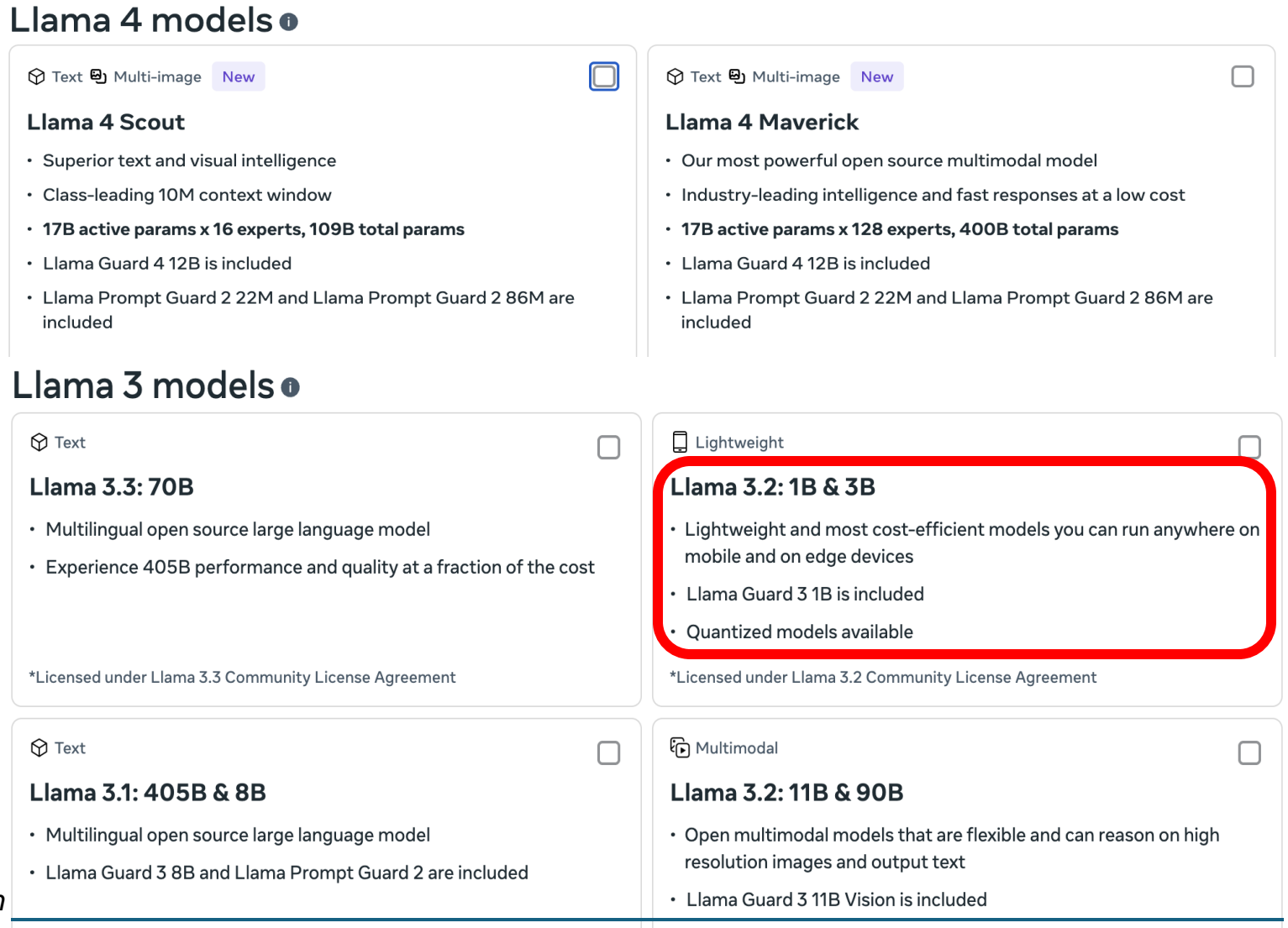


State-of-the-art LLaMA

Modern LLMs are primarily based on the **Transformer** architecture (see *https://arxiv.org/pdf/1706.03762*)

LLaMA developed by **Meta AI** as an open-source alternative to proprietary LLMs e.g. GPT

State-of-the-art LLaMA:



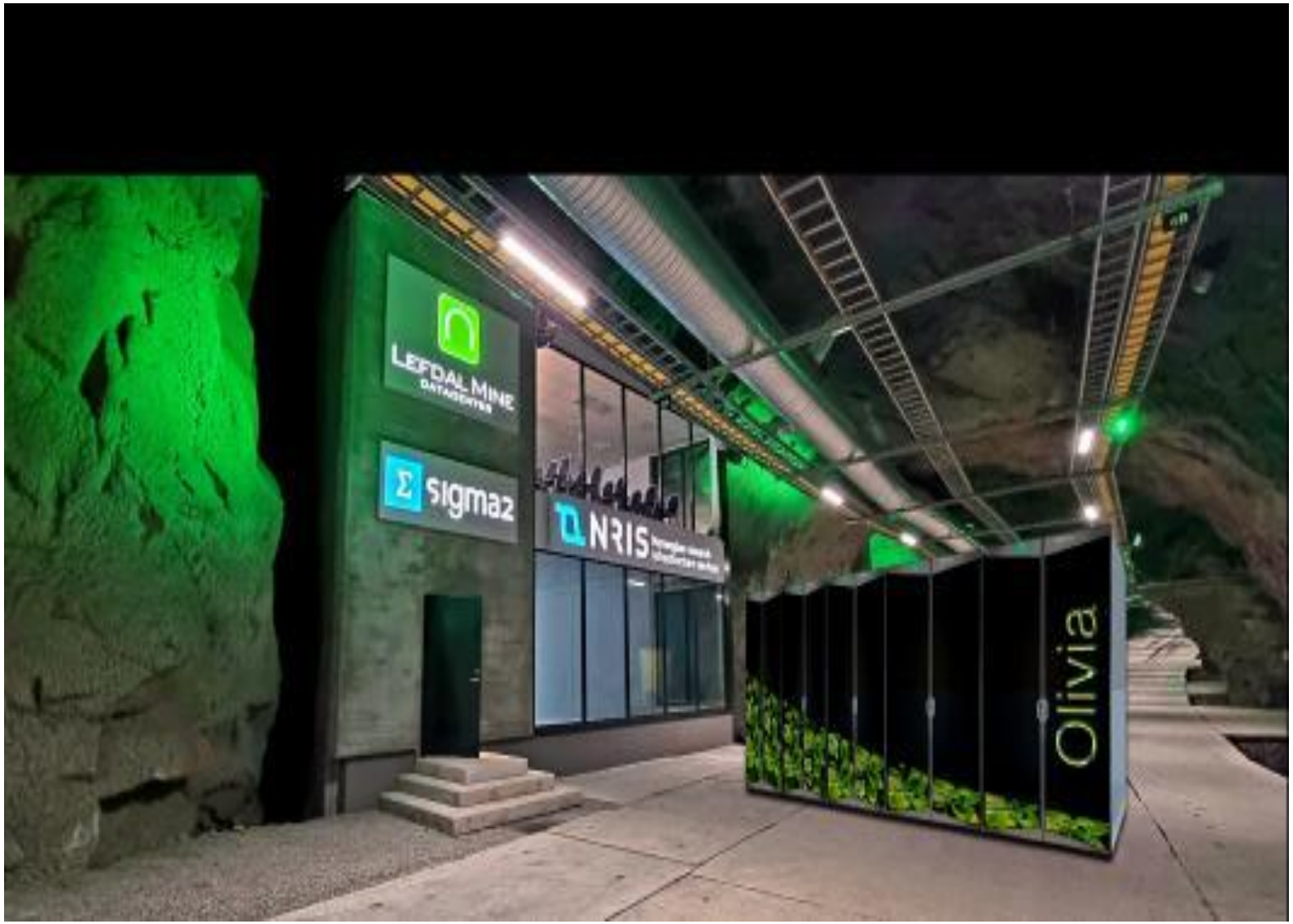
www.llama.com

Environment Setup on Olivia Supercomputer

- **Overview of Olivia supercomputer**
- **SSH setup on Olivia**
- Setting up **PyTorch**, **Torchao**, **Torchtune & peft**
- Verifying **CUDA support** and GPU availability

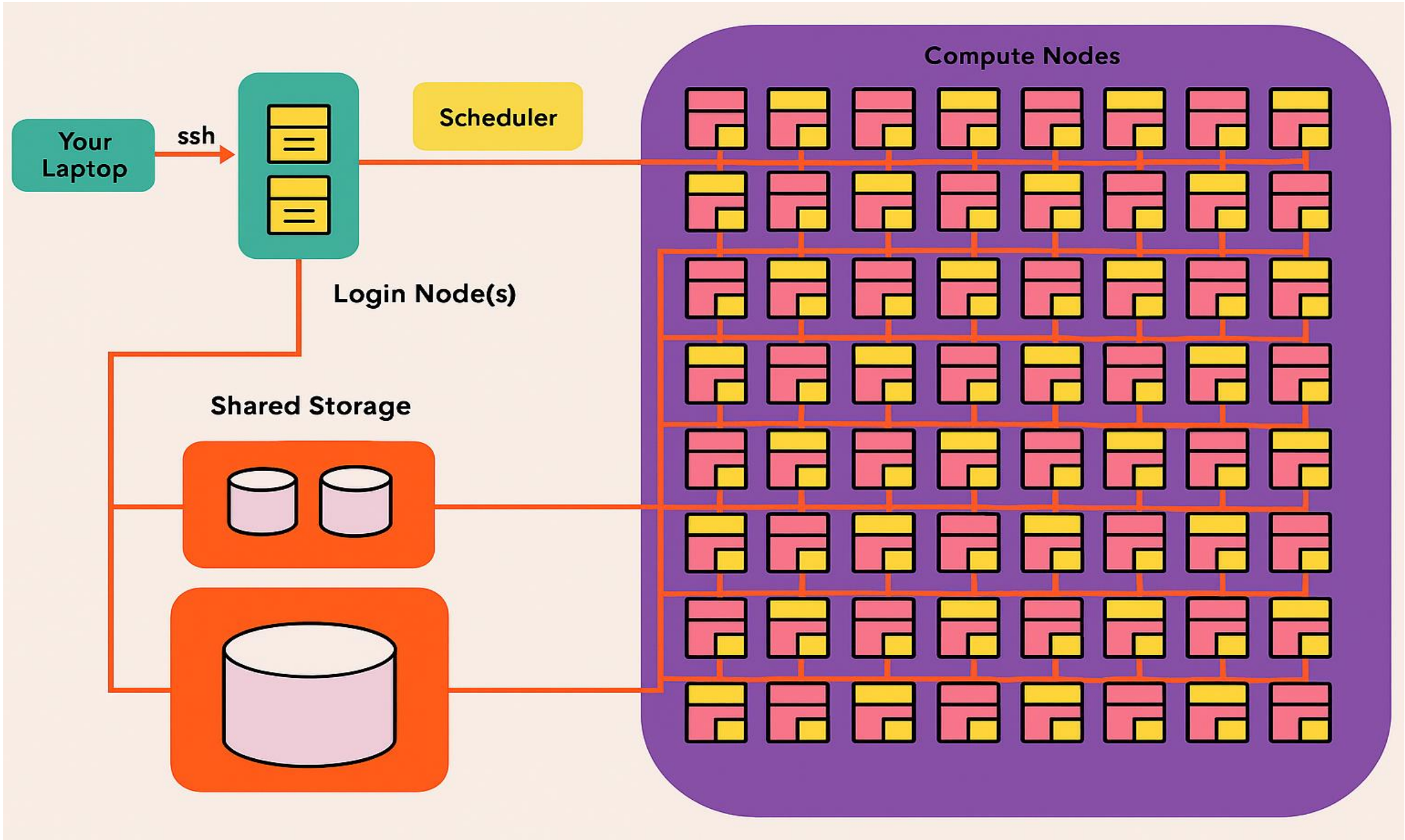
Norway's New Supercomputer - Olivia

- **Olivia** is located in Lefdal Mine (Kjølsdalen)
- System: **HPE Cray** Supercomputing EX
- The GPU partition consists of:
  - 76 GPU-nodes (**304** GPUs)
  - NVIDIA **Grace Hopper Superchip** (**GH200** 96 GB)
  - **HPE Slingshot Interconnect**



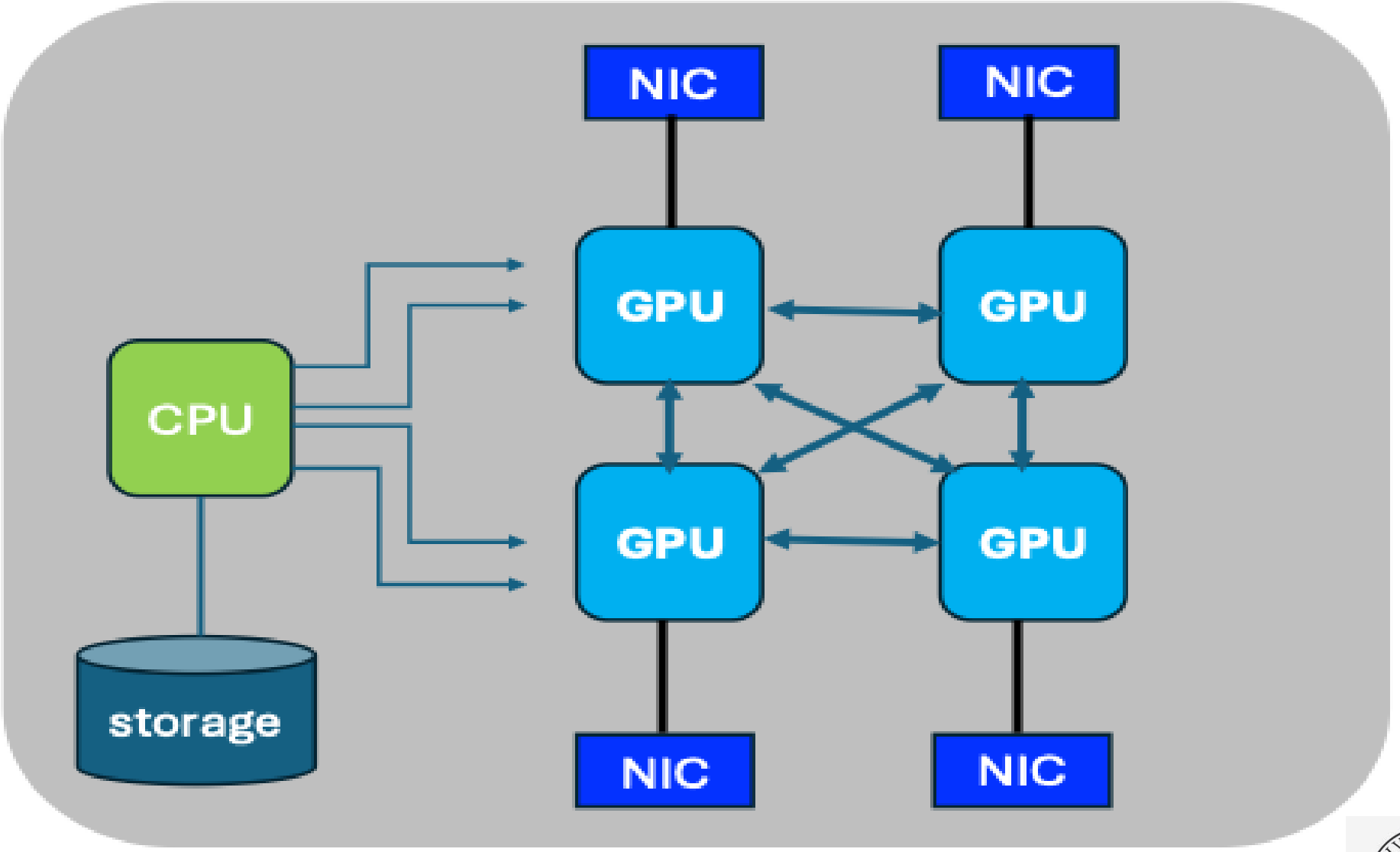
https://www.sigma2.no/news/2024/olivia-becomes-name-norways-next-supercomputer

Basic Cluster-Architecture



Adapted from https://indico.cern.ch/event/814979/contributions/3401193/attachments/1831477/3105158/comp_arch_codas_2019.pdf

Compute node diagram (Simplified)

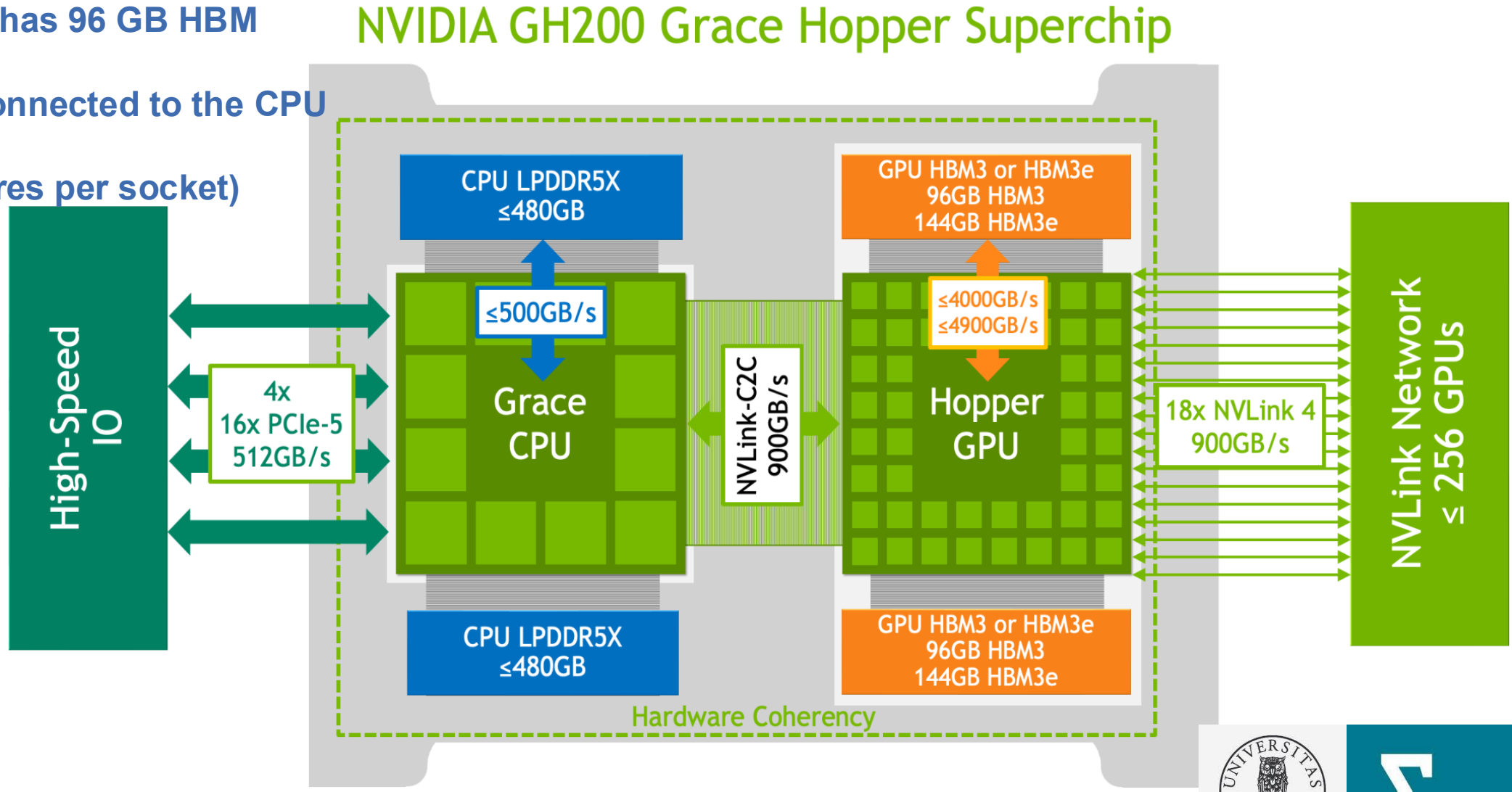


Overview of the Grace Hopper System

Olivia:
Each GH200 device has 96 GB HBM

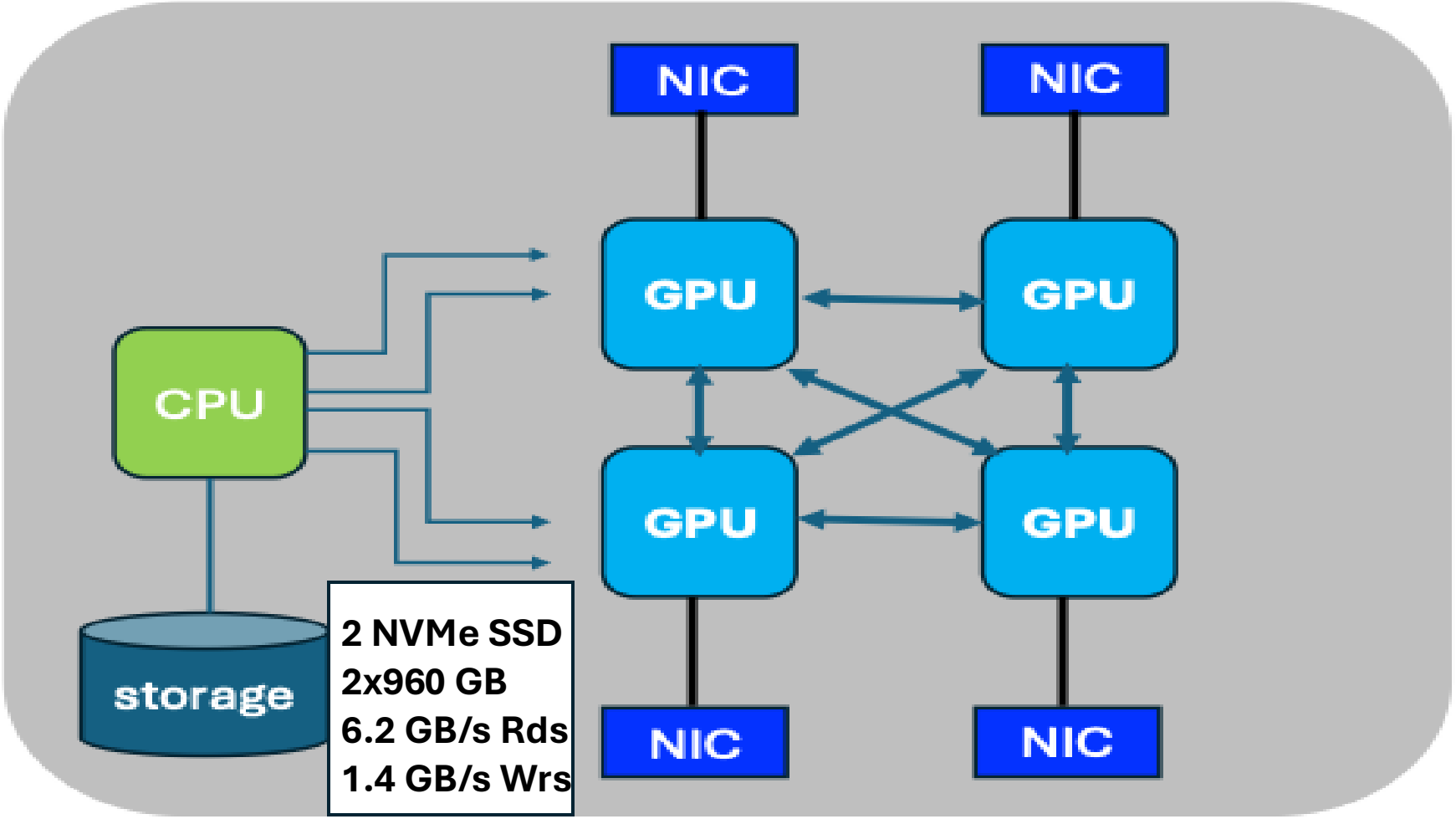
120 GB LPDDR5x connected to the CPU

288 cpu-core (72 cores per socket)



https://resources.nvidia.com/en-us-grace-cpu/nvidia-grace-hopper

Compute node diagram (Simplified)



Divide storage into several filesystems: **/cluster; /cluster/work; /home**; etc
Spaces can be physically or logically separated

```
[hicham@uan02:~> lfs df -h | grep 'filesystem_summary'
filesystem_summary:          2.8P       34.9T        2.8P   2% /cluster
filesystem_summary:          1.0P       92.2T      936.6T   9% /cluster/work
filesystem_summary:       476.4T      356.5G      471.2T   1% /cluster/software
```

GPU-affinity - Olivia system

- Binding a process or thread to a specific GPU.
- Workload is consistently executed on the specified GPU(s).
- Improve performance by reducing data transfer overhead.

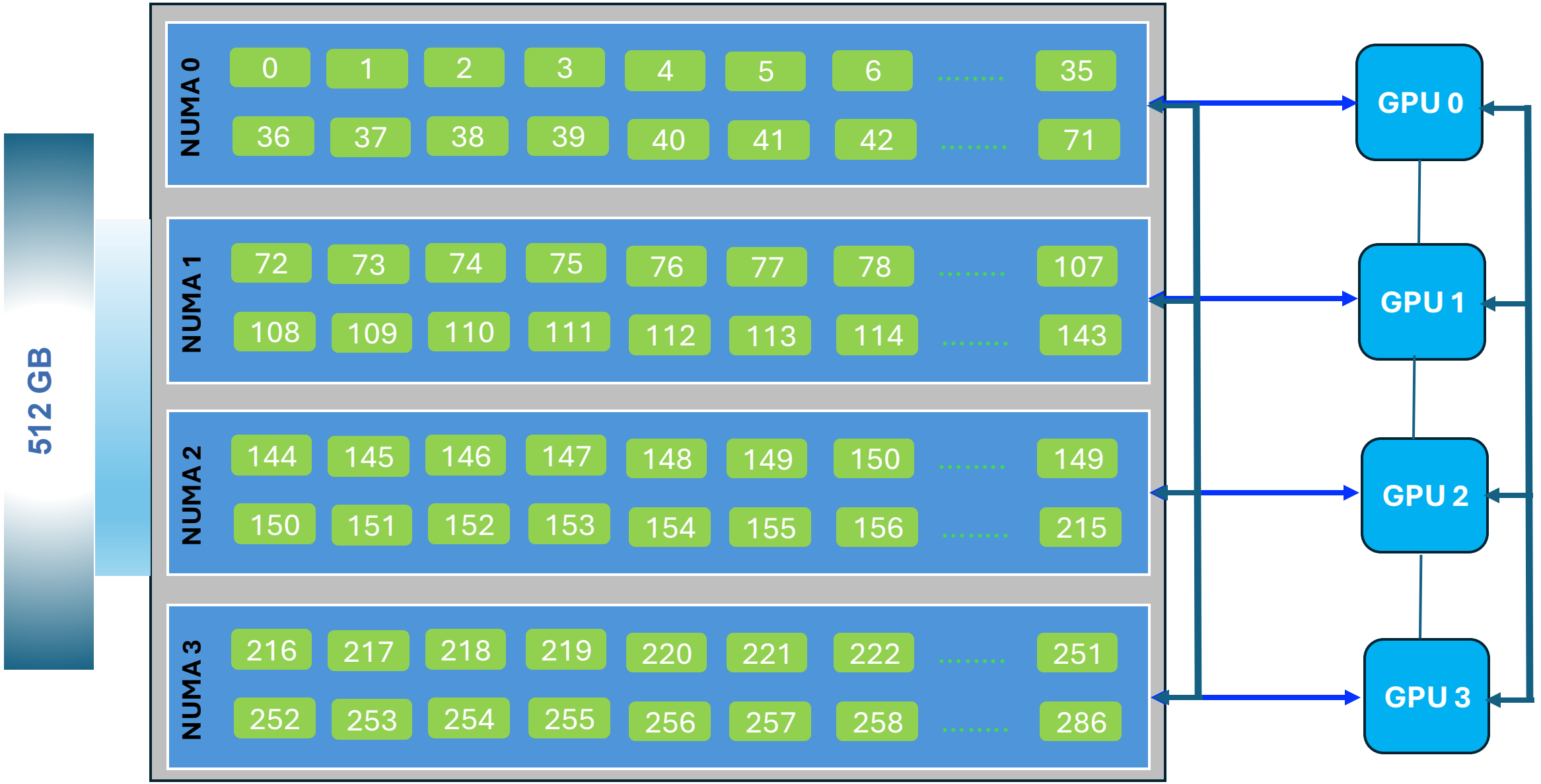
```
[hicham@x1000c0s1b1n0:~> nvidia-smi topo --matrix
        GPU0    GPU1    GPU2    GPU3    CPU Affinity    NUMA Affinity   GPU NUMA ID
GPU0     X      NV6     NV6     NV6     0-71    0               4
GPU1    NV6      X      NV6     NV6     72-143  1               12
GPU2    NV6     NV6      X      NV6     144-215 2               20
GPU3    NV6     NV6     NV6      X      216-287 3               28
```

GPU	CPU Affinity	NUMA Affinity	GPU NUMA ID
GPU0	0–71	0	4
GPU1	72–143	1	12
GPU2	144–215	2	20
GPU3	216–287	3	28

--cpu-bind=map_cpu:... → explicitly binds each task to the CPU cores nearest its GPU.

SLURM will then match **rank 0 → GPU0 (cores 0–71)**, **rank 1 → GPU1 (cores 72–143)**, ... & so on.

GPU-affinity - (simplified diagram)



Environment Setup on Olivia Supercomputer

- **SSH setup on Olivia**

SSH setup to Olivia

```
$ssh username@olivia.sigma2.no
Out:(hicham@login.betzy.sigma2.no) One-time password (OATH) for `hicham':

Out:(hicham@login.betzy.sigma2.no) Password:

$mkdir /cluster/work/projects/nn9970k/$USER


$cd /cluster/work/projects/nn9970k/$USER


$ git clone https://github.com/HichamAgueny/llm-workshop.git

$cd llm-workshop
```

Instructions

llm-workshop / README.md

Hands-On Workflow

1. Explore SLURM job scripts.
2. Review configuration files for LoRA FT.
3. Submit jobs and monitor GPU usage.
4. Inspect logs, metrics, and visualize results.
5. Experiment with hyperparameters (e.g. LoRA rank, dropout).
6. Perform inference on QA & summarization tasks.

Table of Contents

Detailed instructions for each step are provided in the corresponding folder README files:

- Instructions for building customised PyTorch singularity container: container/README.md
- Instructions for testing the built PyTorch container: container/README.md
- Instructions for fine-tuning setup (i.e config. files, recipes etc): container/README.md#fine-tuning-setup
- Instructions for Fine-tuning on a single GPU: fine-tuning-singlegpu/README.md
- Instructions for Fine-tuning on multiple GPUs: fine-tuning-multigpu/README.md
- Instructions for inference: inference/README.md
- Instructions for GPU monitoring and visualizing metrics: tools/README.md
- Instructions for profiling: profiling/README.md
- Guided exercises for practice are described here: exercise/README.md

Environment Setup on HPC (Olivia Supercomputer)

➢Setting up **PyTorch**, **Torchao**, **Torchtune & peft**

•**Instructions for building customized PyTorch singularity container: container/READEME.md**

Environment Setup on HPC - Olivia

Building a customized **pytorch container** with additional packages: **Torchao, Torchtune & peft**

- **Allocate an interactive session on the GPU node** (**accel partition; Olivia**):

A GPU node is made available (temporarily) for building singularity containers

```
$ssh x1000c2s0b0n0
```

Environment Setup on HPC - Olivia

PyTorch singularity Container

https://catalog.ngc.nvidia.com/orgs/nvidia/containers/pytorch/tags

Set temporary directories for **Singularity to redirect Singularity's temporary and cache storage** to **/tmp**

```
$export SINGULARITY_TMPDIR=/tmp/$USER
$export SINGULARITY_CACHEDIR=/tmp/$USER
```

Pull **Singularity container**

```
$singularity pull pytorch2.5_cu2.61_py3.10.sif
 docker://nvcr.io/nvidia/pytorch@sha256:
 931fe4023f51f73bc709f3516c94715b509b4e7
 29d21b59c00e3b890bf4d85b3

$rm -rf /tmp/$USER
```

Contents of the PyTorch container

This container image contains the complete source of the version of PyTorch in `/opt/pytorch`. It is prebuilt and installed in the default Python environment (`/usr/local/lib/python3.10/dist-packages/torch`) in the container image.

The container also includes the following:

- **Ubuntu 22.04** including **Python 3.10**
- **NVIDIA CUDA 12.6.1**
- **NVIDIA cuBLAS 12.6.3.1**
- **NVIDIA cuDNN 9.4.0.58**
- **NVIDIA NCCL 2.22.3**
- NVIDIA RAPIDS™ 24.08
- **rdma-core 39.0**
- NVIDIA HPC-X 2.20
- **OpenMPI 4.1.7**
- GDRCopy 2.3
- TensorBoard 2.16.2
- **Nsight Compute 2024.3.1.2**
- **Nsight Systems 2024.4.2.133**
- **NVIDIA TensorRT™ 10.4.0.26**
- **Torch-TensorRT 2.5.0a0**
- **NVIDIA DALI® 1.41**
- **nvImageCodec 0.2.0.7**
- **MAGMA 2.6.2**
- **JupyterLab 4.2.5** including **Jupyter-TensorBoard**
- **TransformerEngine** 1.10
- TensorRT Model Optimizer 0.15.1

https://docs.nvidia.com/deeplearning/frameworks/pytorch-release-notes/rel-24-09.html

Definition file - Olivia

Customizing the PyTorch Container

A **Singularity definition file** is a recipe for building a container.

It specifies:

- **Bootstrap source** (base image)
- **%post** section for setup commands
- Optional metadata, environment variables, and file additions

```
Bootstrap: localimage

# Base image: Local Singularity image
From: /cluster/work/projects/nn9970k/hicham/llm-workshop/container/pytorch2.5_cu2.6.1_py3.10.sif


%post
# Create /opt/python with symlinks for easy Python access.
mkdir /opt/python
ln -s /usr/bin/python3.10 /opt/python/python3.10
ln -s /opt/python/python3.10 /opt/python/python3
ln -s /opt/python/python3.10 /opt/python/python


# Install additional Python packages inside container
python -m pip install torchtune==0.4.0
python -m pip install torchao==0.12.0
python -m pip install peft==0.17.0
```

```
$ apptainer build --ignore-fakeroot-command pytorch2.5_cu2.6.1_py3.10_custom.sif
  pytorch2.5_cu2.6.1_py3.10_arm.def
```

https://md.sigma2.no/olivia-sw#Build-modified-container

Environment Setup on HPC - Olivia

➢Verifying **CUDA support** and GPU availability

•**Instructions for testing the built PyTorch container:** container/READEME.md#testing-the-container

Environment Setup on HPC - Olivia

Run singularity image

- **Allocate an interactive session on the GPU node (accel partition; Olivia):**

```
$salloc -A NN9970K -t 00:15:00 -p accel -N 1 --gpus 1 --mem-per-cpu 8G
```

- **Launch PyTorch container**

```
$apptainer shell --nv -B $PWD:$PWD pytorch2.5_cu2.6.1_py3.10_custom.sif
```

- **Check CUDA support and other configs.**

```
apptainer> From here run python
python -c "import torch; print(torch.cuda.is_available())"
python -c "import torch; print(torch.__config__.show())"
```

- **Check packages**

```
apptainer> pip list

apptainer> tune ls
```

Fine-Tuning Setup

- Downloading **LLaMA 3.2-1B-Instruct** via Hugging Face
- Generating config. files and recipes
- Understanding and modifying configuration files

•**Instructions for fine-tuning setup (i.e config. files, recipes etc): container/READEME.md#fine-tuning-setup**

Fine-Tuning Setup

What is needed ? **Download** a Llama model, **Copy** Config & Recipe files

- After launching PyTorch container

```
Apptainer>
```

- Download **LLaMA 3.2-1B-Instruct** via e.g. Hugging Face

```
$tune download meta-llama/Llama-3.2-1B-Instruct
  --output-dir /cluster/work/projects/nn9970k/$USER/llm-workshop/data/Llama-3.2-1B-Instruct
  --ignore-patterns "original/consolidated*"
  --hf-token your-hugging-face-token
```

- Copy **CONFIG** file

```
  $tune cp llama3_2/1B_lora_single_device .
  $tune cp llama3_2/1B_lora_multi_device .
  $tune cp generation .
  Change path: /tmp/Llama-3.2-1B-Instruct
To:/new-Path
  sed -i "s|/tmp|/new-Path|g" 1B_lora_single_device.yaml
```

- Fine-tuning on a single GPU
- Fine-tuning on multiple GPUs
- Inference

- Copy built-in **RECIPE** file (python script)

```
  $tune cp lora_finetune_single_device .
  $tune cp lora_finetune_distributed .
  $tune cp generate .
```

- Fine-tuning on a single GPU
- Fine-tuning on multiple GPUs
- Inference

Hands-on (10 min):

A customized PyTorch container is available at your work area

/cluster/work/projects/nn9970k/$USER/llm-workshop/container

Environment Setup on Olivia

- Instructions for testing the built PyTorch container: container/READEME.md#testing-the-container

Fine-Tuning Setup

- Instructions for fine-tuning setup (i.e config. files, recipes etc): container/READEME.md#fine-tuning-setup

N.B. You don't need to download the pretrained weights. They are already available at:

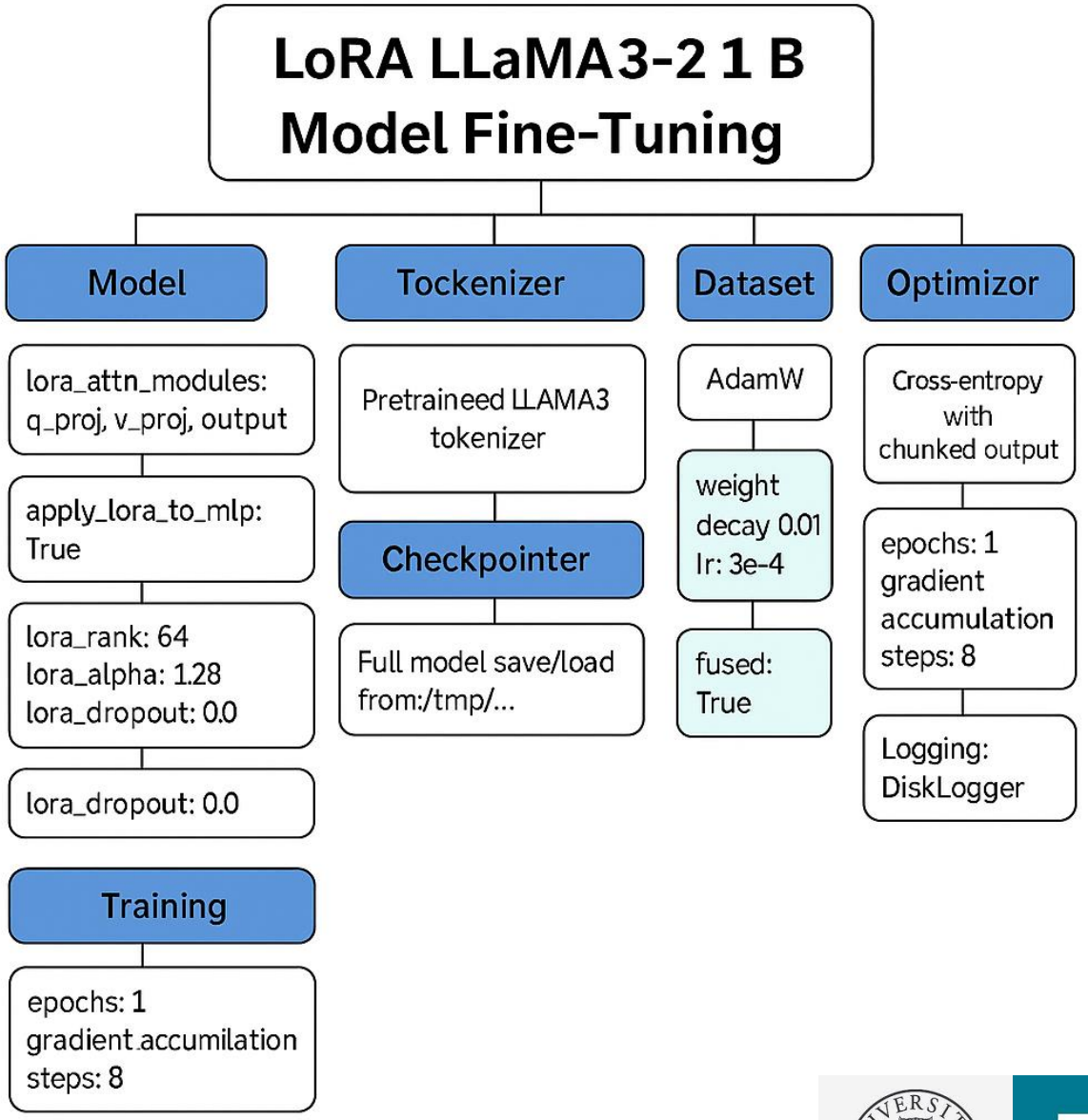
/cluster/work/projects/nn9970k/$USER/llm-workshop/data/Llama-3.2-1B-Instruct

LLaMA3-2 1B **Fine-Tuning** Config Overview

Single GPU

Due to limited computational resources:

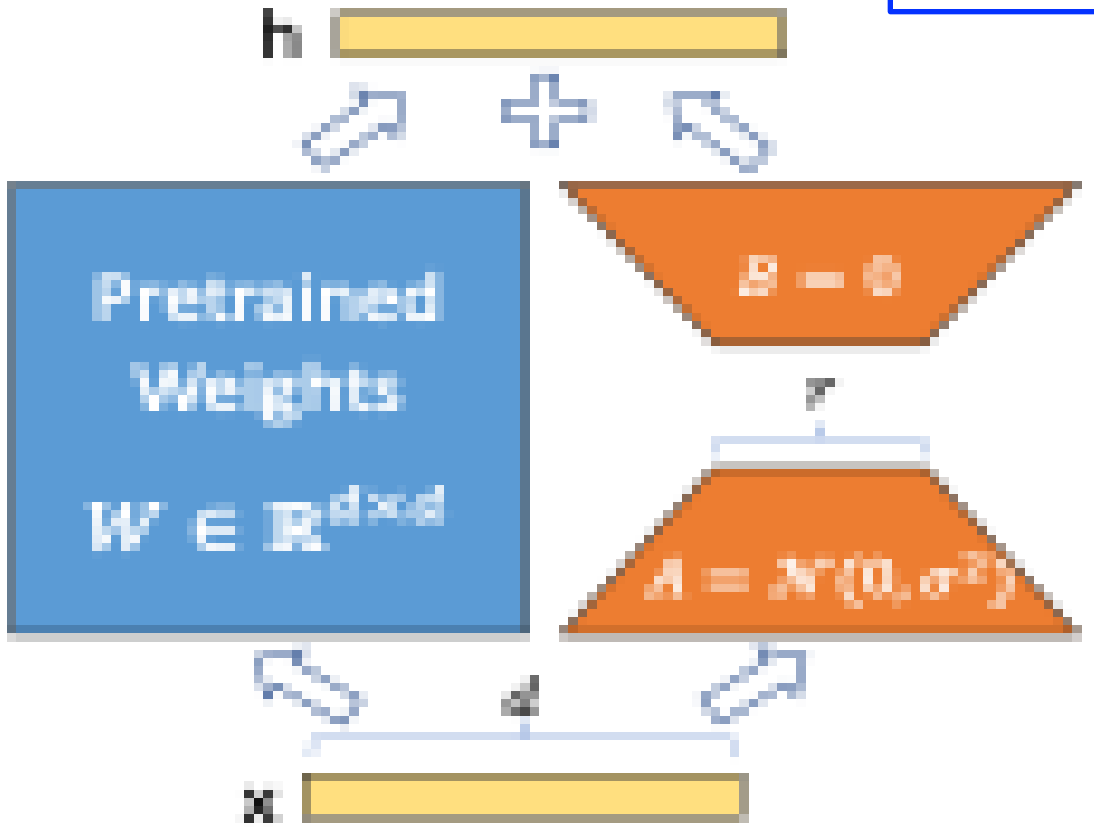
- **Light model: 1B parameter of LLaMA3-2**
- **LoRA techniques for fine-tuning**



LoRA Overview

LoRA: Low-Rank Adaptation of Large Language Models

Edward Hu* **Yelong Shen*** **Phillip Wallis** **Zeyuan Allen-Zhu**
Yuanzhi Li **Shean Wang** **Lu Wang** **Weizhu Chen**
Microsoft Corporation
{edwardhu, yeshe, phwallis, zeyuana,
yuanzhil, swang, luw, wzchen}@microsoft.com
yuanzhil@andrew.cmu.edu
(Version 2)



Modified forward pass

$$h = W_0 x + \Delta W x = W_0 x + B A x.$$

Random Gaussian initialization for A and zero for B

$\Delta W = BA$ is zero at the beginning of training

LoRA https://arxiv.org/pdf/2106.09685

Model Arguments

```
model:
  _component_: torchtune.models.llama3_2.lora_llama3_2_1b
```

- Loads the LLaMA3-2 1B model with **LoRA** integration.

It's a **1B parameter variant of LLaMA3.2** (Meta's LLaMA3 architecture)

```
  lora_attn_modules: ['q_proj', 'v_proj', 'output_proj']
```

- LoRA is applied to specific transformer attention components: **query, value, and output projections**.

Model Arguments

```
model:
  _component_: torchtune.models.llama3_2.lora_llama3_2_1b
```

- Loads the LLaMA3-2 1B model with **LoRA** integration.

It's a **1B parameter variant of LLaMA3.2** (Meta's LLaMA3 architecture)

```
  lora_attn_modules: ['q_proj', 'v_proj', 'output_proj']
```

- LoRA is applied to specific transformer attention components: **query, value, and output projections**.

```
  apply_lora_to_mlp: True
```

- Enables **LoRA** for the model's MLP (feedforward) layers too, not just attention. (apply_lora_to_mlp: False: only to attention layers)

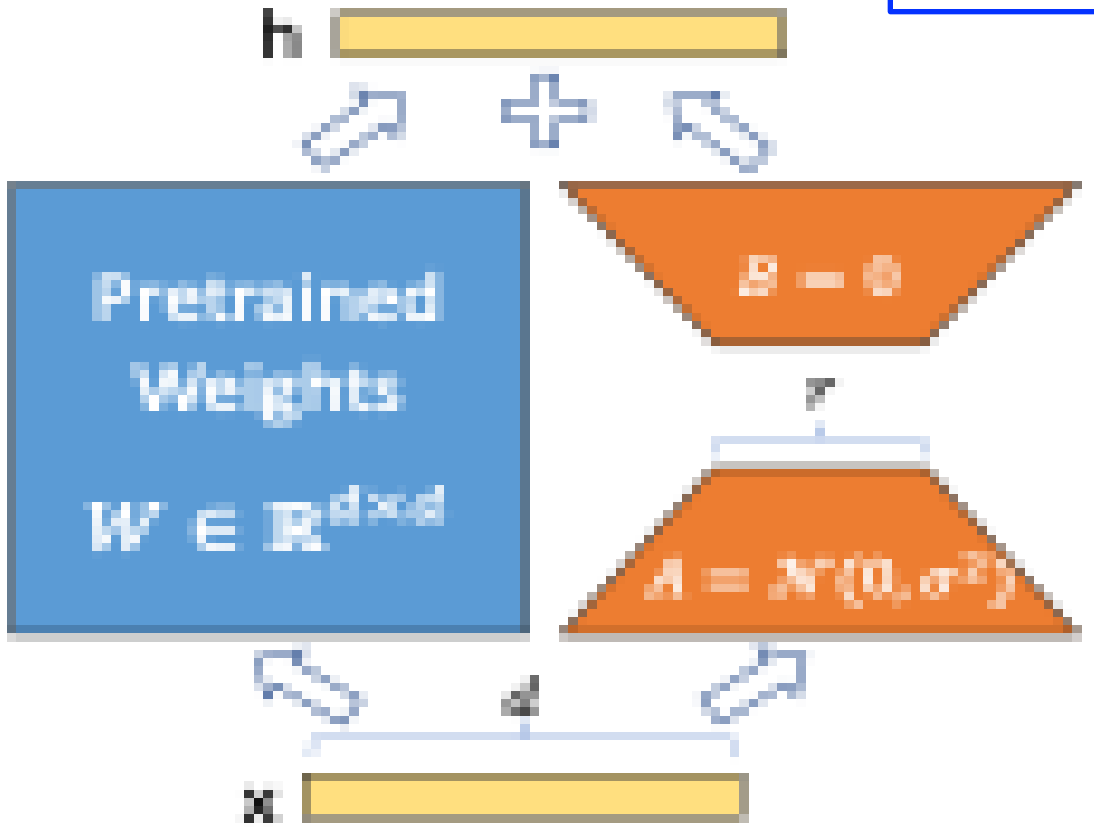
```
  lora_rank: 64
  lora_alpha: 128
  lora_dropout: 0.0
```

- **lora_rank** determines low-rank dimensionality of matrices **A** and **B** that are added to the pre-trained model.
- **lora_alpha** scales LoRA updates (commonly 2×rank).
- **lora_dropout** is optional and is turned off (can be tuned if overfitting is noticed).

LoRA Overview

LoRA: Low-Rank Adaptation of Large Language Models

Edward Hu* **Yelong Shen*** **Phillip Wallis** **Zeyuan Allen-Zhu**
Yuanzhi Li **Shean Wang** **Lu Wang** **Weizhu Chen**
Microsoft Corporation
{edwardhu, yeshe, phwallis, zeyuana,
yuanzhil, swang, luw, wzchen}@microsoft.com
yuanzhil@andrew.cmu.edu
(Version 2)



Modified forward pass

$$h = W_0 x + \Delta W x = W_0 x + B A x.$$

Random Gaussian initialization for A and zero for B

$\Delta W = BA$ is zero at the beginning of training

LoRA https://arxiv.org/pdf/2106.09685

Tokenizer

```
tokenizer:
  _component_: torchtune.models.llama3.llama3_tokenizer
  path: /tmp/Llama-3.2-1B-Instruct/original/tokenizer.model
  max_seq_len: null
```

- Specifies a tokenizer built for LLaMA 3 models.
- Defines the path to the tokenizer model file.
- Specifies the maximum number of tokens the tokenizer shou
- **null** means no limit is enforced here (will be managed elsewl

Checkpointer

```
checkpointer:
  _component_: torchtune.training.FullModelHFCheckpointer
  checkpoint_dir: /tmp/Llama-3.2-1B-Instruct/
  checkpoint_files: [model.safetensors]
  recipe_checkpoint: null
  output_dir: /tmp/Llama-3.2-1B-Instruct/
  model_type: LLAMA3_2
resume_from_checkpoint: False
save_adapter_weights_only: False
```

- Class used for checkpointing.
- Saving and restoring the full Hugging Face model.

- Directory **where checkpoints are read from.**
- Tells the checkpointer **which specific files to load.**
- Loading a training recipe or configuration from a checkpoint.
- Directory to **save** the checkpoints.
- Specifies the model type Llama-3.2-1B-Instruct.
- False: Starts a new training from pre-trained base otherwise it con from where it previously stopped.
- False: Entire model will be saved including LoRA weights.
- True: only **LoRA/adapter weights** are saved. (It will require a separate step to merge them later with the base model at inference).

Key	Meaning
checkpoint_dir	Where checkpoints live (read/write)
checkpoint_files	File(s) to load (here: model.safetensors)
recipe_checkpoint	Optional training state checkpoint (not used here)
output_dir	Where to save new checkpoints
model_type	Tells the system how to parse model weights
resume_from_checkpoint	Whether to continue from a saved state
save_adapter_weights_only	Whether to save only LoRA weights or the full model

Dataset & Sampler

```
dataset:
  _component_: torchtune.datasets.alpaca_cleaned_dataset
  packed: False
seed: null
shuffle: True
batch_size: 4
```

- Dataset of instruction-following demonstrations based on OpenAI's text-davinci-003

- Indicates whether sequences are *packed* for training.
- **False**: Padding is used to ensure consistent sequence lengths within a batch.

- **True**: Input examples are packed together into a continuous stream, which can increase training speed by reducing padding overhead.

- No specific random seed is set.

- Dataset is shuffled to prevent the model from overfitting.

- Number of samples to process in **one step.**

Optimizer & Scheduler

```yaml
optimizer:
  _component_: torch.optim.AdamW
  fused: True
  weight_decay: 0.01
  lr: 3e-4
```

- Specifies the optimizer class: AdamW is widely used in Transformer training.
- Enables a **fused backend implementation of the optimizer** (speeds up training).
- Helps prevent overfitting by penalizing large weights.
- This is the initial learning rate.

```yaml
lr_scheduler:
  _component_: torchtune.training.lr_schedulers.get_cosine_schedule_with_warmup
  num_warmup_steps: 100
```

- Defines how the learning rate changes over time.
- Uses a built-in TorchTune utility to get a **cosine decay** schedule with warmup.
- During warmup, the LR gradually increases from zero to the target LR.
- After warmup, the LR follows a **cosine decay**, slowly decreasing over time (effective for transformer models)
- Number of steps to gradually increase the learning rate before cosine decay begins.

Loss Function

```yaml
loss:
  _component_: torchtune.modules.loss.CEWithChunkedOutputLoss
```

- Specifies the loss function used during training.
- **Cross-Entropy loss** designed to work with chunked model outputs.

Training Config

```
epochs: 1
max_steps_per_epoch: null
gradient_accumulation_steps: 8
compile: False
```

- Sets the total number of full passes over the training dataset.
- Sets the optional limit on how many steps to run **per epoch**.
- **Null:** means it will run over the **entire dataset** for each epoch.
- Sets how many steps before applying an optimizer update.
- Useful for a limited GPU memory. Number of forward + backward passes to accumulate gradients **before updating weights**.
- Allows simulating a **larger batch size** without increasing GPU memory usage.
- **False**: Disables torch.compile()
- **True**: Enables torch.compile(), which can speed up and optimize model execution.

Logging

```
output_dir: /tmp/lora_finetune_output
metric_logger:
  _component_: torchtune.training.metric_logging.DiskLogger
  log_dir: ${output_dir}
log_every_n_steps: 1
log_peak_memory_stats: True
```

- Specifies the directory where outputs (logs, checkpoints, metrics, etc.) will be saved

- **DiskLogger** writes logs to disk (Useful for later analysis or visualization).
- Sets the log directory to be the same as **output_dir**: /tmp/lora_finetune_output
- Logs metrics **after every single training step.**
- **True:** Enables logging of **peak GPU memory usage.**
- (useful for debugging memory issues or profiling training efficiency)

Environment

```
device: cuda
dtype: bf16
```

- Specifies that training will run on a **CUDA-compatible GPU.**
- Sets the computation **data type** to **bfloat16**, a lower-precision format.
- **(**16-bit floating-point format**).**
- Less precise than fp32 (32 bit) but more than fp16 (16 bit)**.**

Activation Memory Control

```
enable_activation_checkpointing: False
enable_activation_offloading: False
```

Memory optimization Flags:

- **False**: Default. All activations checkpointings are stored (faster training, higher memory usage).

- Moves activations from **GPU to CPU** memory during training.
- **True**: Reduces GPU memory usage, but slows training. Useful for **very large models** that don't fit entirely on GPU.

Profiler Configuration

```
profiler:
  _component_: torchtune.training.setup_torch_profiler
  enabled: False
```

- Profile training performance (CPU/GPU utilization, memory, bottleneck
- False: means no profiling occurs.

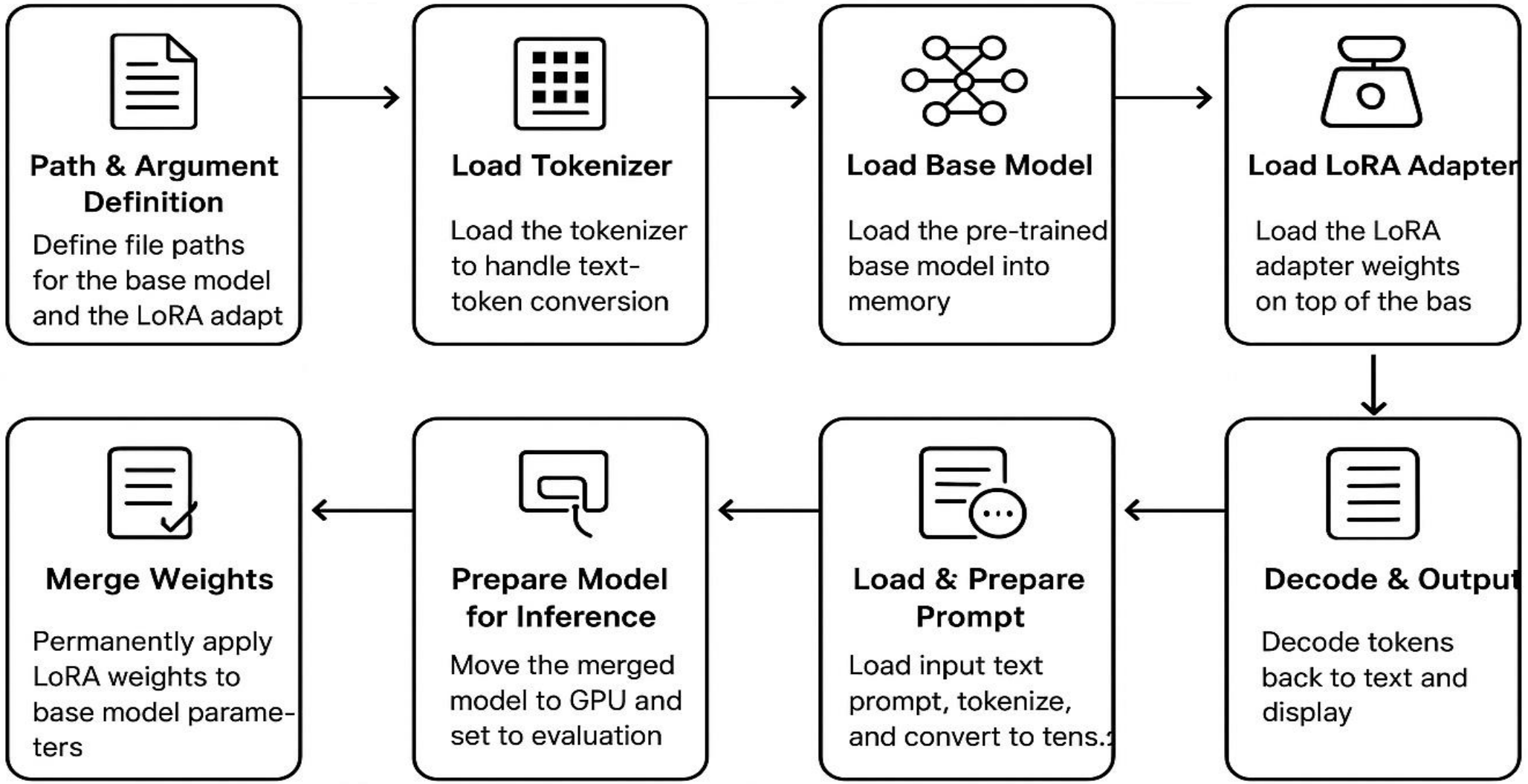
```
output_dir: ${output_dir}/profiling_outputs  # Saves traces (e
```

- Specifies the directory where profiling outputs are saved.

```
cpu: True   # Profile CPU operations
cuda: True  # Profile GPU (CUDA) operations
```

- Enables tracking of operations on CPU & GPU devices.

Structure of the Inference process



Hands-on:

Fine-Tuning on a single GPU + Inference

Dataset: Alpaca dataset

Question Answering tasks

Hands-on: Fine-Tuning on a single GPU + Inference

Dataset: Alpaca dataset: Question Answering (QA) tasks

```
dataset:
  _component_: torchtune.datasets.alpaca_cleaned_dataset
  packed: False
seed: null
shuffle: True
batch_size: 4
```

- **Hands-On Workflow**

Instructions for Fine-tuning on a single GPU: fine-tuning-singlegpu/README.md

Instructions for inference: inference/README.md

1.Explore SLURM job scripts.

2.Review configuration files for LoRA FT.

3.Submit jobs and monitor GPU usage.

4.Inspect logs, metrics, and visualize results.

5.Perform inference on QA task.

Hands-on:

Fine-Tuning on multiple GPUs + Inference

Dataset: Alpaca dataset

Question Answering tasks

Hands-on: Fine-Tuning on multi GPUs + Inference

Dataset: Alpaca dataset: Question Answering (QA) tasks

```
dataset:
  _component_: torchtune.datasets.alpaca_cleaned_dataset
  packed: False
seed: null
shuffle: True
batch_size: 4
```

- **Hands-On Workflow**

Instructions for Fine-tuning on a single GPU: fine-tuning-multigpu/README.md

Instructions for inference: inference/README.md

1.Explore SLURM job scripts.

2.Review configuration files for LoRA FT.

3.Submit jobs and monitor GPU usage.

4.Inspect logs, metrics, and visualize results.

5.Compare metrics: single GPU vs multiple GPUs

Hands-on:

Profiling: Fine-Tuning on a single GPU

Profiling: Fine-Tuning on a single GPU

Extracted from:
1B_lora_single_device_QA_profiling.yaml

```
# Profiler (enabled)
profiler:
  _component_: torchtune.training.setup_torch_profiler
  enabled: True

  # Output directory for trace artifacts
  output_dir: /cluster/projects/nn9970k/$USER/llm-workshop/data/profiling_outputs

  # Activities to trace
  cpu: True
  cuda: True

  # Trace options
  profile_memory: True
  with_stack: False
  record_shapes: True
  with_flops: True

  # Scheduling options
  wait_steps: 5
  warmup_steps: 3
  active_steps: 2
  num_cycles: 1
```

Profiling: Fine-Tuning on a single GPU

- **Hands-On Workflow**

Instructions for profiling: profiling/READEME.md

1.Explore SLURM job scripts.

2.Review configuration files for profiling.

3.Submit a job.

4.Visualize results.

Memory Profiling: Fine-Tuning on a single GPU

Hands-on:

Fine-Tuning on a Single GPU + Inference

Dataset: Xsum dataset

Summarization tasks

Hands-on: Fine-Tuning on a single GPU/multiGPUs + Inference

Dataset: Xsum dataset: Summarization tasks

```
dataset:
  _component_: torchtune.datasets.alpaca_cleaned_dataset
  packed: False
seed: null
shuffle: True
batch_size: 4
```

- **Hands-On Workflow**

Instructions for Fine-tuning on a single GPU & multi GPUs: exercise/README.md

1.Explore SLURM job scripts.

2.Review configuration files for LoRA FT.

3.Submit jobs and monitor GPU usage.

4.Inspect logs, metrics, and visualize results.

5.Perform inference on Summarization task.

Wrap up & Discussion

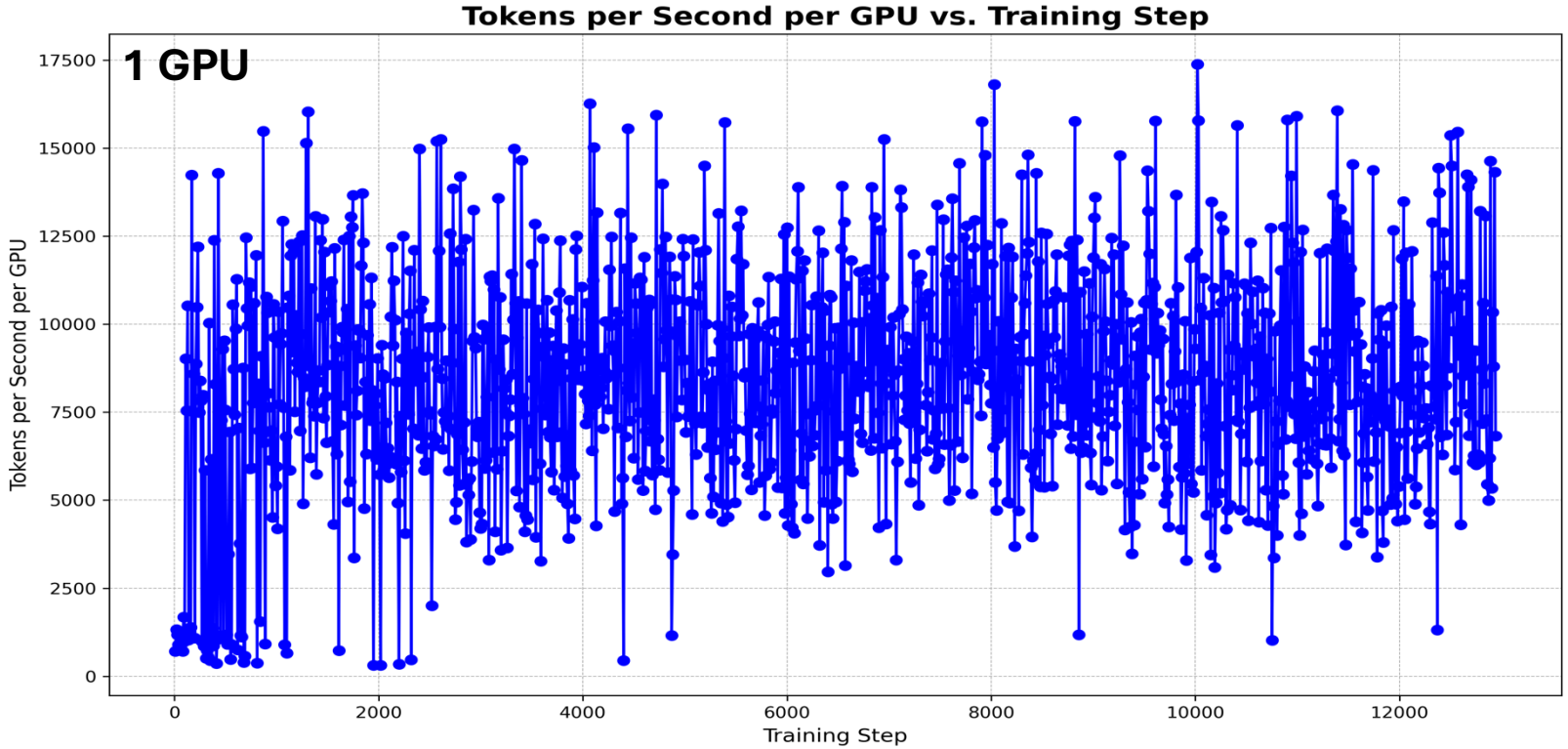
Throughput per-GPU:

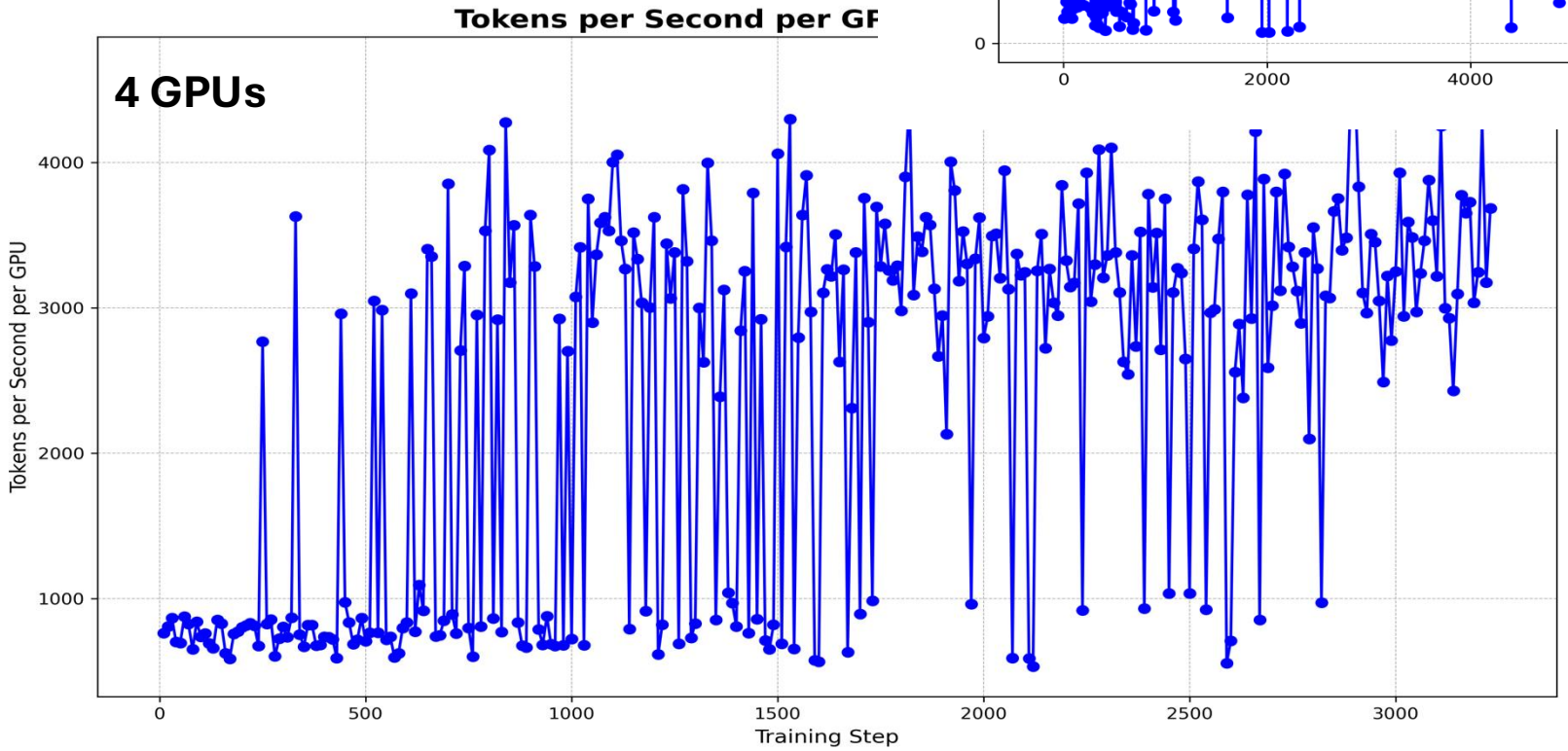
Model: **Llama 3.2-1B**

1GPU: up to **13000** Tokens per sec per GPU
4 GPUs: up to **4000** Token per sec per GPU

Total system throughput : 4x4000=16000

Runtime (1 GPU): 0.12 s/training step
Runtime (4 GPUs): 0.10 s/training step





Parallel efficiency of ≈ 30%

1B model, doesn't benefit from multi-GPU training

A single GPU can easily fit the model in memory and process batches efficiently.

Additional overheads in multiGPU setup

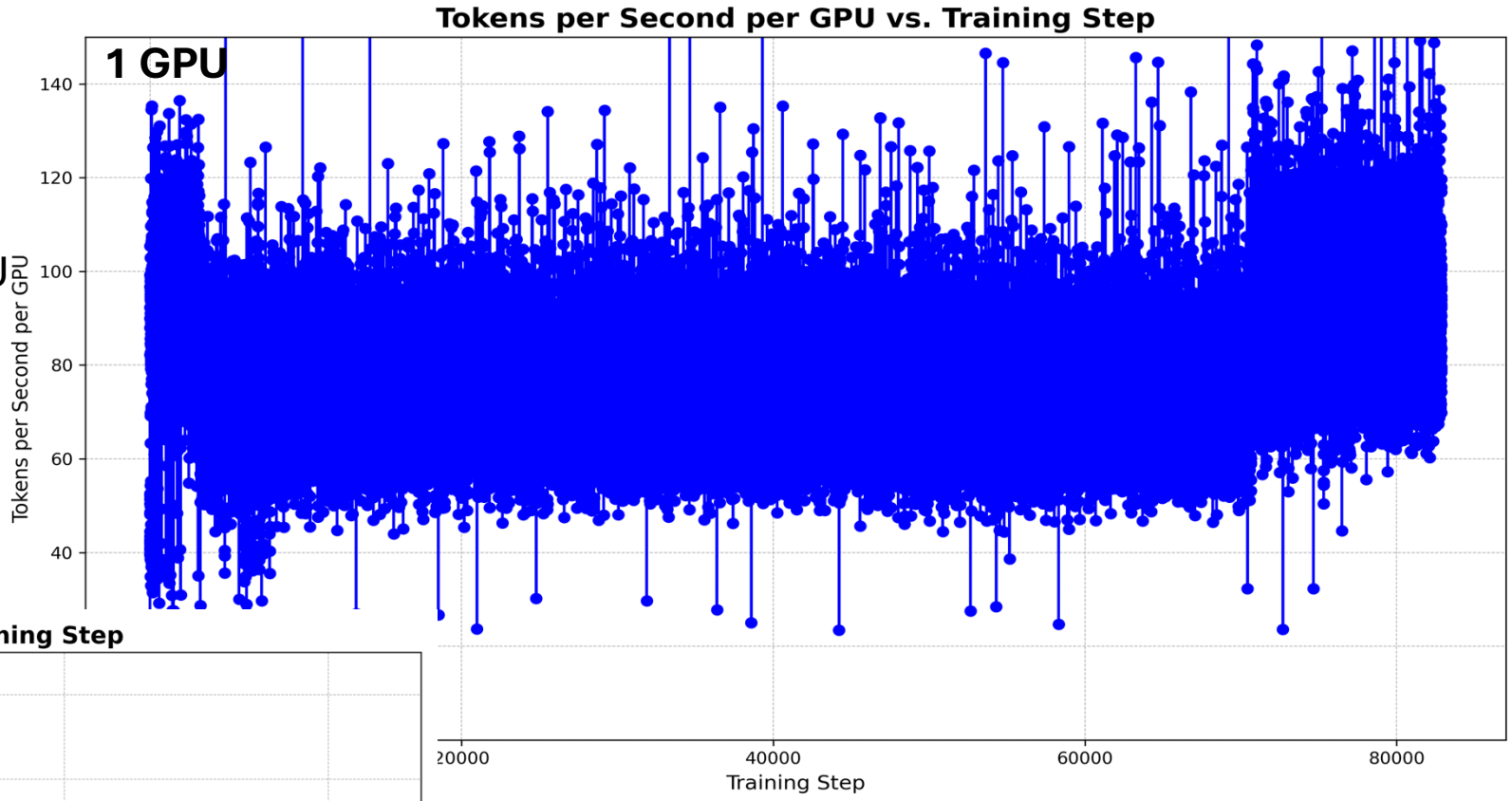
Throughput per-GPU:

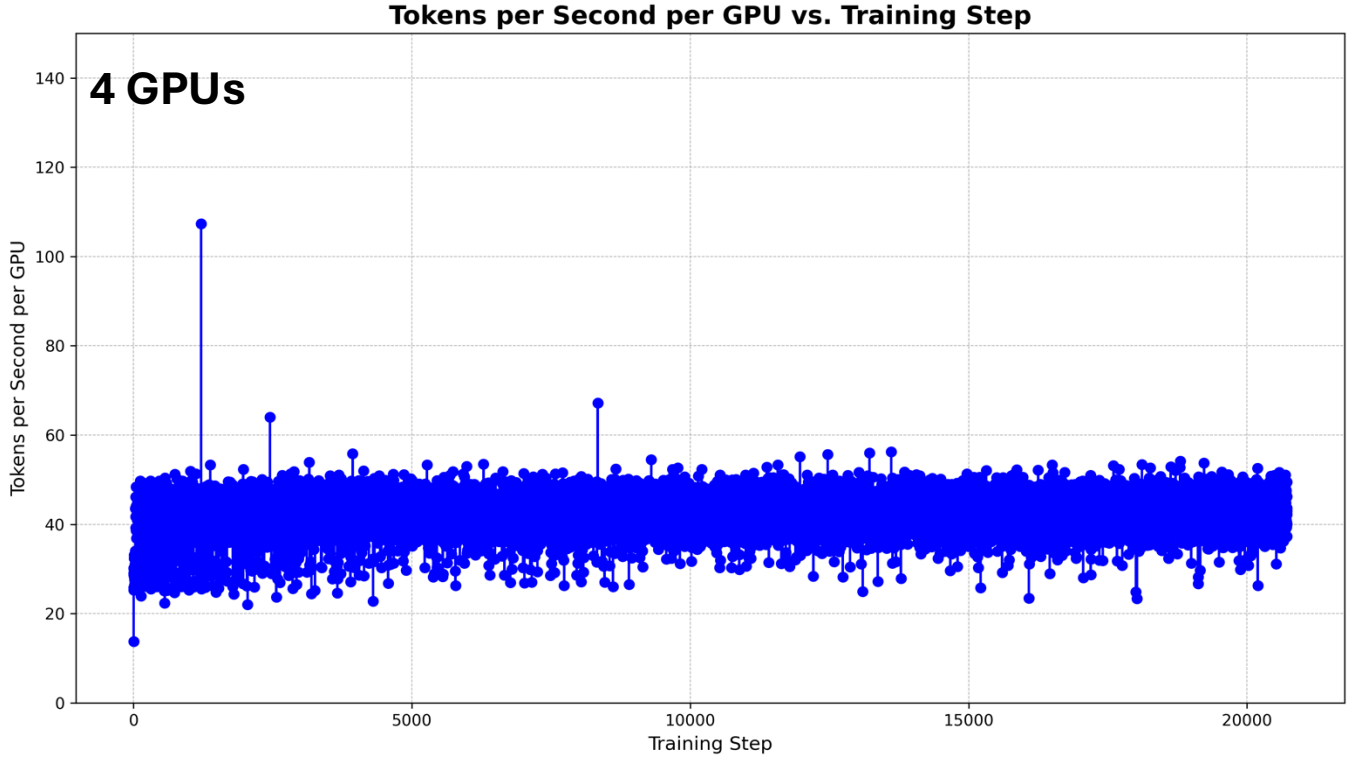
Model: **Llama 3.2-11B**

1GPU: up to **100-120** Tokens per sec per GPU
4 GPUs: up to **50** Token per sec per GPU

Total system throughput : 4x50=200

Runtime (1GPU): 1512m33s(**25.20 h**)
Runtime (4 GPUs): 542m17s (**9.04 h**)





Scaling efficiency:
Parallel efficiency of ≈ 70%

Using **4 GPUs** reduced the training time
by a **factor of 2.8**

For much larger models, where the model itself cannot fit on a single GPU.....

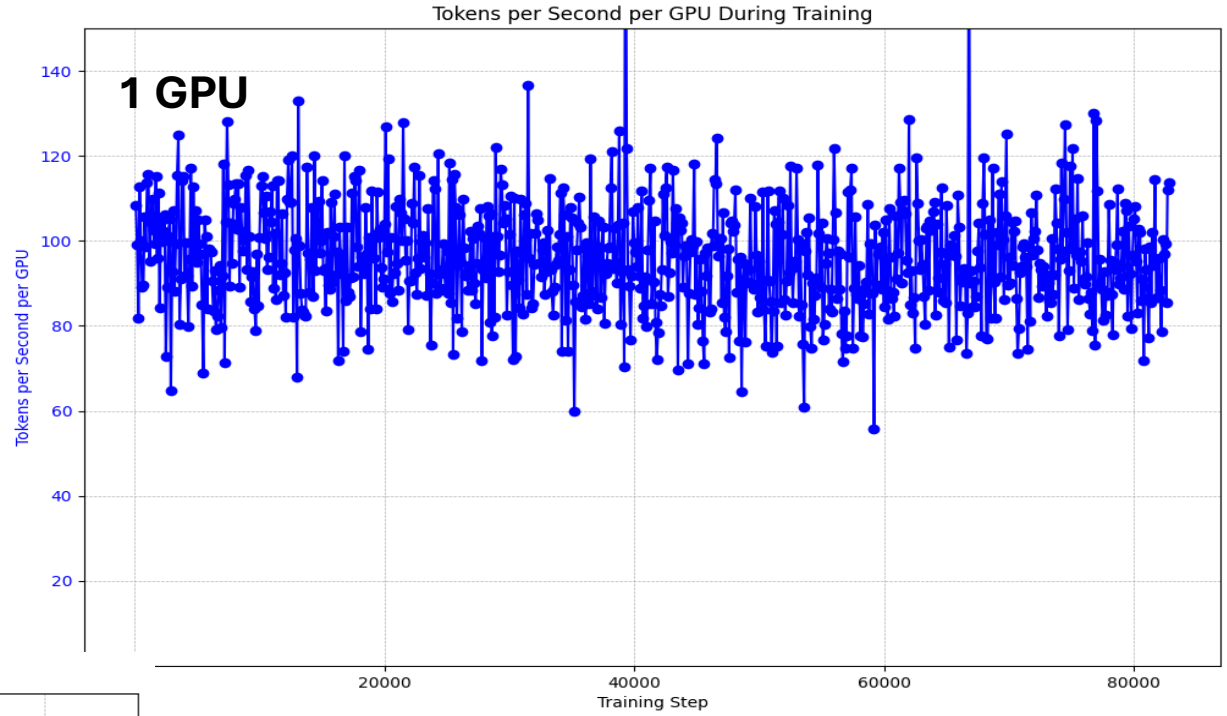
Throughput per-GPU:

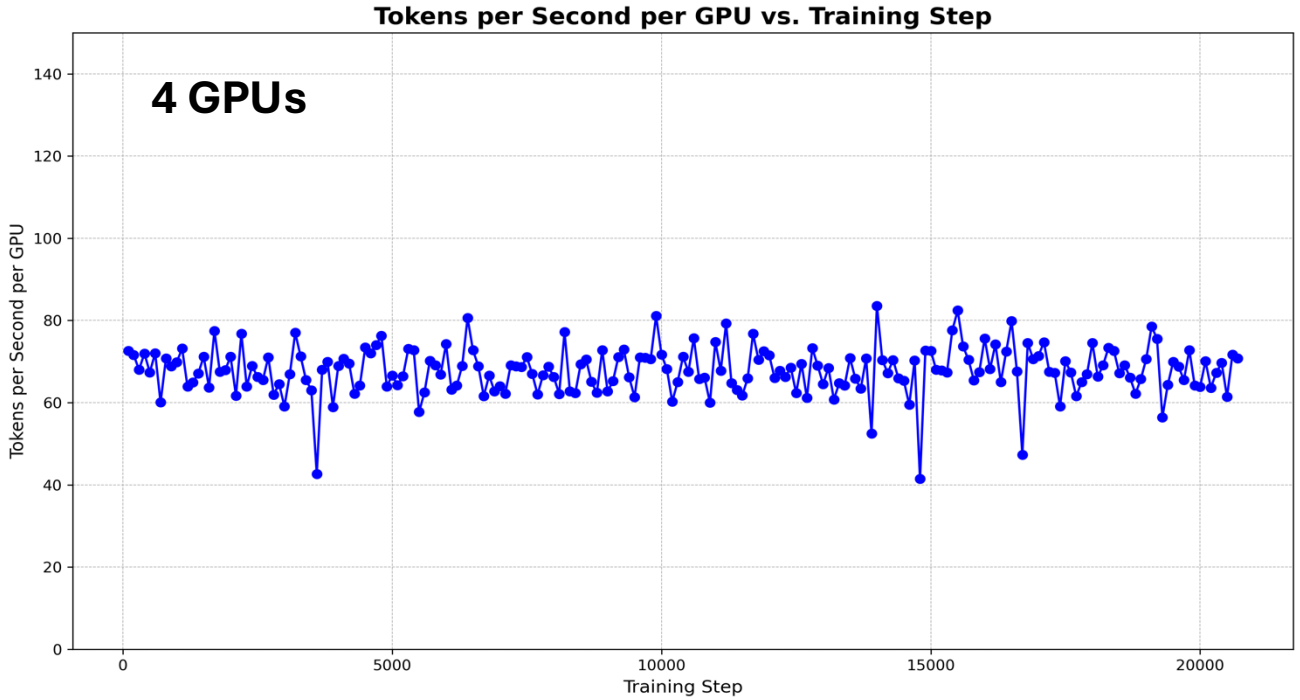
Model: **Llama 3.2-11B**

1GPU: up to **110** Tokens per sec per GPU
4 GPUs: up to **70** Token per sec per GPU

Total system throughput : 4x70=280

Runtime (1GPU): 1.1 second/training step
Runtime (4 GPUs): 0.4 s/training step





Scaling efficiency:
Parallel efficiency of ≈ 70% on 4 GPUs

Using **4 GPUs** reduced the training time
by a **factor of 2.8**

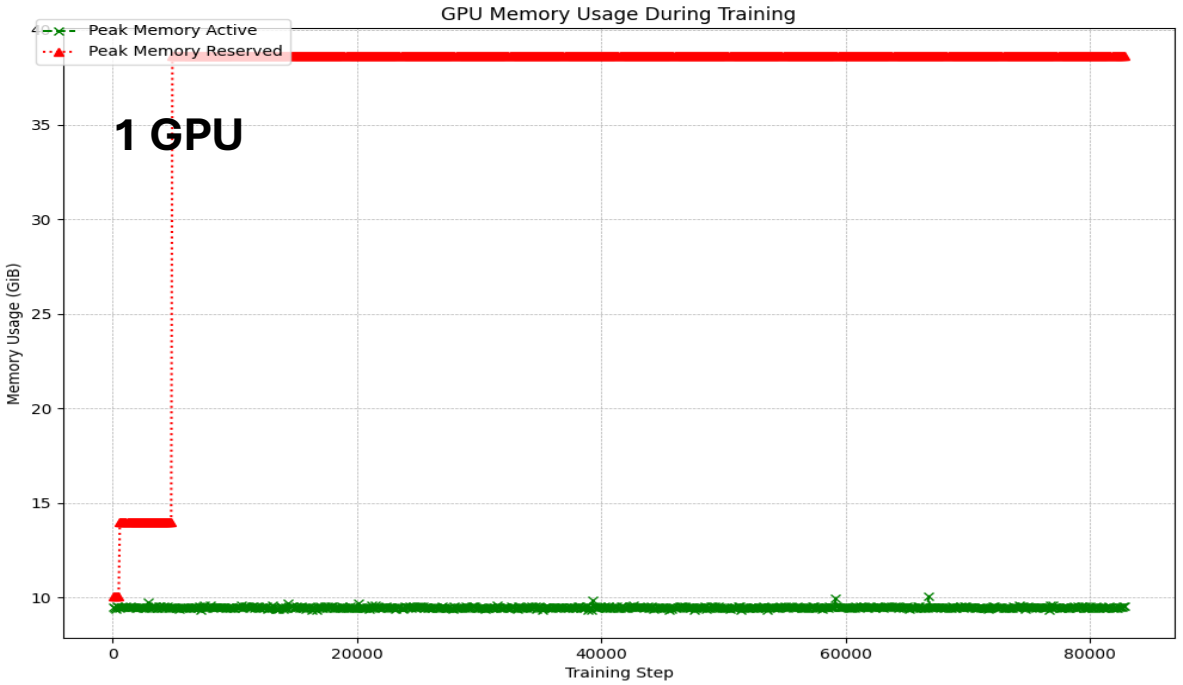
For much larger models, where the model itself cannot fit on a single GPU.....

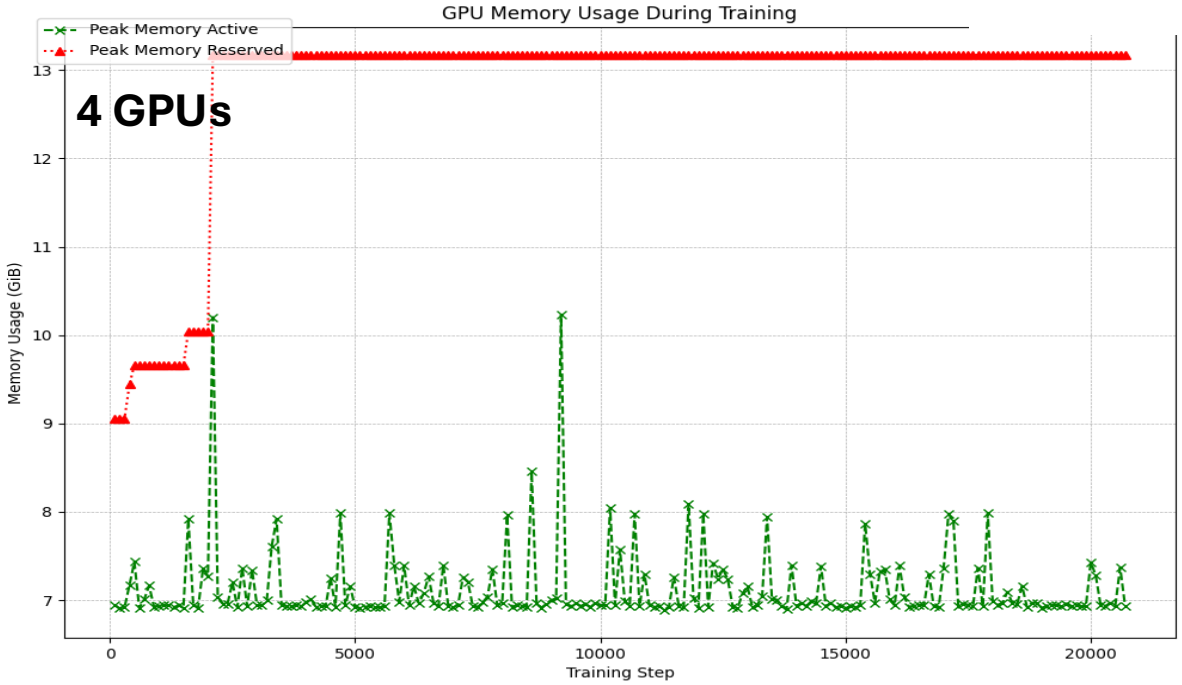
Memory per-GPU:

Model: **Llama 3.2-11B**

1 GPU: Peak mem active is **9-10 GiB**
peak mem reserved is up to **38.6 GiB**

4 GPUs: Peak mem active is **7-8 GiB**
Peak mem reserved is up to **13.2 GiB**





Moving from **1 GPU to 4 GPUs reduces** the **peak reserved memory** by a **factor of 3**.
(but slightly lowering the peak active memory)

Efficient distribution of memory across the GPUs.

What’s Next ?

Planned Workshop on AI & related topic

To include e.g.:

- **DoRA and other optimization techniques**
- **Pre-processing of data**
- **Model Evaluation**
- **Distributed training across multiple nodes**

References

- **GitHub repo LLM-workshop:** https://github.com/HichamAgueny/llm-workshop
- **LoRA:** https://arxiv.org/pdf/2106.09685
- **LoRA Dropout:** https://arxiv.org/pdf/2404.09610
- **PEFT:** https://arxiv.org/abs/2312.12148
- **Ultimate Guide to Fine-Tuning LLMs**: https://arxiv.org/pdf/2408.13296
- **Merging model**: https://huggingface.co/docs/peft/en/developer_guides/lora#weight-decomposed-low-rank-adaptation-dora
- **Transformer:** https://arxiv.org/pdf/1706.03762
- **Fastformer:** https://arxiv.org/pdf/2108.09084
- **LlaMA models:** www.llama.com
- **PyTorch container**: https://docs.nvidia.com/deeplearning/frameworks/pytorch-release-notes/rel-24-09.html
- **Olivia supercomputer:** https://www.sigma2.no/meet-olivia-norways-next-supercomputer
- **Overview of High-Performance Computing (HPC):** https://github.com/HichamAgueny/HPC_architecture/blob/main/webinar_HPC.pdf
- **Distributed Training in HPC systems**: https://github.com/HichamAgueny/AI-guide/blob/main/DistributedTraining_Guide.md

Defining hyperparameter

- A **hyperparameter** is a configurable variable set before training an AI model.
- It controls the learning process such as the **learning rate**, **batch size**, number of **epochs**.
- Its tuning is crucial for enhancing the model's performance and accuracy.

Hyperparameter	Description	Impact on Training
Learning Rate	Controls how much model weights are updated during training. Used in optimization algorithms like SGD.	**- Smaller values**: More stable but slower training. **- Larger values**: Faster updates but risk of overshooting minima or instability.
Batch Size	Number of training samples processed before updating model weights.	**- Smaller batches**: More updates, may generalize better but noisier. **- Larger batches**: More stable updates, but require more memory.
Epochs	One complete pass through the full training dataset.	**- More epochs**: Better learning up to a point; excessive epochs may cause overfitting. - **Fewer epochs**: May lead to underfitting.